

Intention-aware Monitoring of Dynamic Targets with Bounded Uncertainty and Minimum Fleet

Shuaikang Wang¹, Jianyang Xue¹, Yiannis Kantaros² and Meng Guo¹

Abstract—Long-term monitoring of large areas with a minimum number of agents is a prerequisite for many applications such as surveillance, search and rescue. **Fleets of unmanned aerial and ground vehicles have been deployed to monitor numerous dynamic targets in a collaborative way.** It is a challenging problem requiring online task coordination, information fusion, and motion control. The majority of existing works either assume a passive all-to-all observation model to minimize the summed uncertainties of all targets, or optimize over the joint discrete actions while neglecting the dynamic constraints of the agents and unknown behaviors of the targets. Instead, the agents may be restricted to actively track only a limited number of targets simultaneously, while the targets move within a road network to accomplish unknown high-level temporal tasks. This work proposes an integrated and distributed task and motion coordination algorithm for long-term monitoring tasks with a minimum fleet, which consists of: (I) the online Bayesian inference of task-level intention of each target modeled as temporal formulas; (II) the distributed communication and coordination algorithm based on local Nash stability, given the intention-aware Kalman prediction of target trajectories with uncertainty measure; (III) the event-triggered nonlinear model predictive controller to optimize the agent trajectories, subject to the uncertainty bounds of its assigned targets. It is shown that the estimation uncertainty for all targets is explicitly bounded at all time; the inference of target intention converges to the actual tasks; and the agents switch between active and inactive roles online. The proposed methods are validated via large-scale simulations of up to 50 agents and targets under various configurations.

I. INTRODUCTION

Fleets of unmanned aerial vehicles (UAVs) and unmanned ground vehicles (UGVs) have been deployed to monitor areas that are otherwise too large or hostile for humans, e.g., to map time-varying source fields [1], to track moving targets [2], and to patrol and detect intruders [3]. Compared with single agents, task efficiency can be improved significantly via proper assignment and concurrent execution. As a fundamental technique in these applications, the active monitoring of numerous dynamic targets has attracted significant attention [4], [5]. It remains challenging for both task planning and motion control, since monitoring tasks must be coordinated online while the agent trajectories adapt in real time to target behaviors that are only observed online and difficult to predict beforehand.

Furthermore, as shown in Fig. 1, the following quest often occurs from a practical perspective: *how to monitor a large number of dynamic targets with an upper-bounded uncertainty*

The authors are with ¹the School of Advanced Manufacturing and Robotics, Peking University, Beijing 100871, China; and ²the Department of Electrical and Systems Engineering, Washington University at St. Louis, St. Louis, MO, 63130, USA. This work was supported by ¹the National Natural Science Foundation of China (NSFC) under grants 62203017, U2241214, T2121002; ¹the Fundamental Research Funds for the central universities; and ²the ARL grant DCIST CRA W911NF-17-2-0181. Contact: meng.guo@pku.edu.cn.

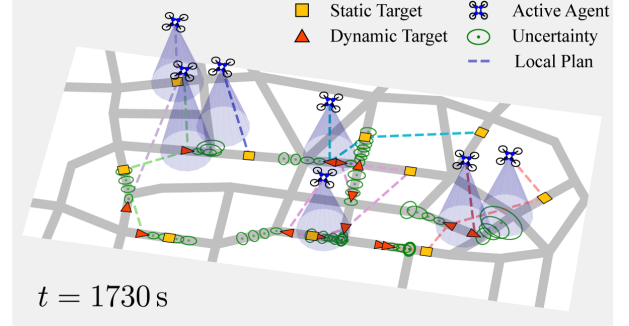


Fig. 1. Illustration of the considered scenario: 25 UAVs of which 7 are active (in blue) are deployed to monitor 15 dynamic targets (in red) and 10 static targets (in yellow). The objective is to design a distributed coordination and control strategy, such that (I) the average number of active agents is minimized; (II) the uncertainty of dynamic targets is bounded explicitly; and (III) the visiting interval of static targets are less than specified. Note that the trajectory and intention of dynamic targets are unknown.

via a minimum number of agents. In other words, the objective is to extend the longevity of the multi-agent fleet while ensuring the monitoring quality. This formulation differs from most existing work [2], [4], [6], which minimizes the summed uncertainty of all targets without explicit bounds and often requires multiple agents per target. In addition, since the target behaviors are unknown a priori, e.g., the path within the road network cannot be determined beforehand, the assignment of targets to agents should adapt to online measurements. Thus, a distributed and scalable coordination method is desired for large fleets and many targets. Lastly, the targets may move accomplish high-level tasks, e.g., transporting goods or surveilling regions of interest. Inferring such task hypotheses can improve the prediction of future behaviors.

A. Related Work

As the most relevant, the work in [4], [6] addressed the problem of active information acquisition (AIA) with sensing agents. The proposed solutions exploit the separation principles under the linear observation model and utilize an offline search algorithm over the joint state-and-information space, via the forward value iteration (FVI) and later the reduced forward value iteration (RVI). This line of methods has been extended to multi-agent fleets, where an optimal, non-myopic, and centralized sampling-based solution is proposed in our earlier work [6]; decentralized myopic planners can be found in [7], [8]. **Recently, this line of research has been enriched by incorporating heterogeneous limited-range sensors for distributed tracking [9], as well as proposing homotopic information gain for sparse active target tracking [10]. Recent work [2] utilizes graph neural networks to learn distributed and scalable action policies from the optimal planner, yet its**

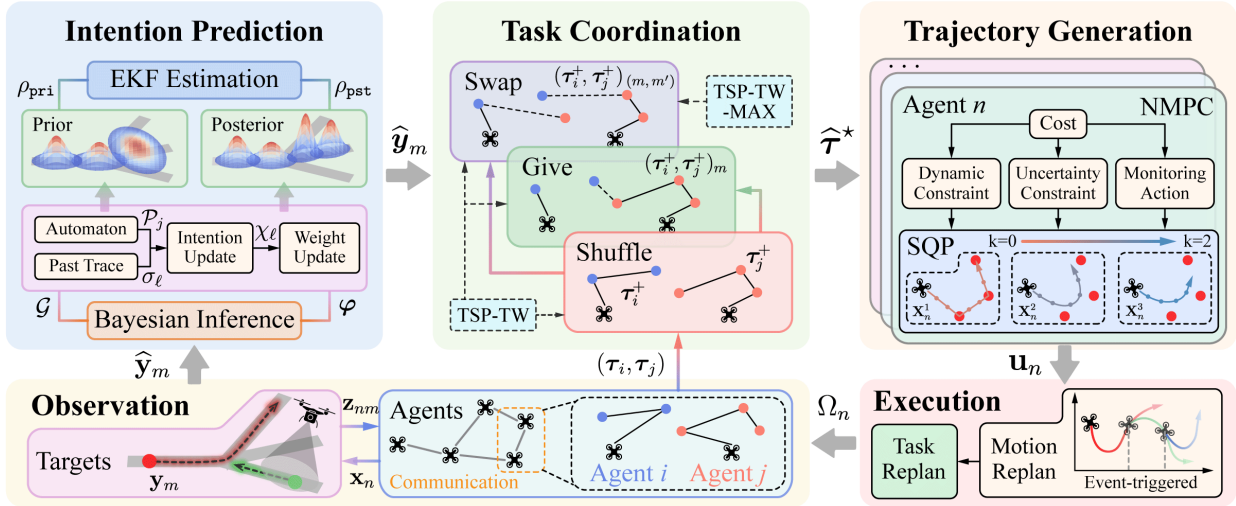


Fig. 2. Overview of the proposed framework, which includes the observation for dynamic targets (**Bottom-Left**); intention prediction via Bayesian inference and EKF estimation (**Top-left**); distributed task coordination through shuffle, give, and swap operations (**Center**); trajectory generation via NMPC with dynamic, uncertainty, and monitoring constraints solved using SQP (**Top-right**); and execution with event-triggered task and motion replanning (**Bottom-right**).

fundamental objective is to minimize the summed uncertainty using a fixed-size fleet, lacking the mechanism to infer the long-term intentions of the targets or minimize the deployed fleet size. However, most of these works assume a fixed team size, discretized control inputs, and a linear observation model; more importantly, the targets move with known dynamics and control inputs. Instead, the dynamic targets considered here have unknown velocities and paths within the road network, thus requiring online prediction and tracking.

On the other hand, another line of research formulates the multi-target monitoring problem as a sequential high-level assignment problem, where agents are assigned to targets for reducing uncertainty. Recent work [11] proposes a general task planning framework for coordinating multi-agent systems under uncertainties and failures. Nevertheless, this framework focuses on the formal task execution and constraints of the agents themselves, rather than actively inferring the long-term temporal intentions of uncooperative dynamic targets or minimizing the required fleet resources. Moreover, numerous works [12], [13] reformulate it as a simultaneous action and target assignment (SATA) problem, and propose distributed approximation algorithms such as the linear programs. However, the target trajectories are often assumed to be given, and the agents move synchronously with a set of pre-defined motion primitives. Recent work [14] studies robust assignment strategies against communication and sensing limitations or attacks. Energy constraints are addressed in [15] with periodic monitoring requirements. However, the combination of dynamic and static targets within a road-network has less results, and the necessity for minimizing the number of deployed agents has been largely overlooked.

Lastly, the importance of predicting the trajectory and intention of dynamic targets when monitoring their motion has been signified in recent work [16]–[18]. The predominant application is autonomous driving, where the intention of surrounding vehicles or pedestrians is essential for predicting future trajectories and hence the control policy. Data-driven predictive control allows real-time observations to be

incorporated into the online optimization of robust control policies [18], [19]. Moreover, various neural network-based tracking methods are proposed such as Conditional Variational Autoencoder (CVAE) [16], long short term memory (LSTM) network [17], and recent diffusion policies capable of multi-modal tracking without prior knowledge [20]. However, these approaches mainly focus on *short-term and real-time* prediction of future trajectories, instead of long-term prediction on the task level. The synergy between online inference of long-term task hypotheses and the model-based prediction of trajectories remains relatively under-explored.

Our previous work [21] considers a similar problem formulation, but relies on a centralized solution that can be vulnerable to communication failures. It neither takes into account the prediction of target intentions, nor how it could be utilized to improve the state estimation and task coordination.

B. Our Method

To tackle these issues, this work proposes an integrated and distributed task and motion coordination algorithm, for the long-term monitoring problem of dynamic targets, as shown in Fig. 2. The proposed solution consists of four main components: (I) an automaton-based inference algorithm that updates the task hypotheses of each target in a Bayesian way from the timed sequence of observed features; (II) an EKF-based and intention-aware state estimator that predicts the target trajectory online, with distinct prior and posterior updates on roads and at intersections; (III) a fully distributed online coordination algorithm that assigns timed monitoring tasks subject to the predicted target trajectories and uncertainties, ensuring a Nash-stable assignment and a minimum fleet size with finite-iteration convergence; and (IV) a distributed event-triggered nonlinear model predictive control (ENMPC) algorithm that incorporates the agent dynamics and the predicted target behaviors. Extensive simulations are performed over large-scale fleets and complex scenes against several baselines.

Main contribution of this work is three-fold: (I) the novel formulation of active monitoring problem for dynamic tar-

gets with unknown temporal intentions, subject to an upper-bounded uncertainty and a minimum fleet. To the best of our knowledge, this problem has not been addressed in related work; (II) the proposed integrated task and motion coordination algorithm that adapts the fleet size and the agent trajectories online, under real-time measurements of the targets and inference of intention hypotheses; (III) the fully-distributed and scalable solution that allows the deployment of a few UAVs to monitor a larger number of dynamic targets.

II. PROBLEM DESCRIPTION

A. Fleet of Agents

Consider a team of N autonomous agents (such as UAVs) that coexist within a workspace $\mathcal{W} \subset \mathbb{R}^3$. Without loss of generality, each agent $n \in \mathcal{N} \triangleq \{1, \dots, N\}$ follows the same generic nonlinear dynamics:

$$\mathbf{x}_n(t+1) \triangleq f(\mathbf{x}_n(t), \mathbf{u}_n(t)), \quad (1)$$

where $f(\cdot)$ is the dynamic model; $\mathbf{x}_n(t) \in \mathcal{X}$ and $\mathbf{u}_n(t) \in \mathcal{U}$ are the state and control at time step $t \geq 0$. It is assumed that the agents can communicate over a distributed and connected communication network $\mathcal{G}_c(t)$ at all time, such that information from one agent can be routed to any other agent.

B. Dynamic Targets with Temporal Tasks

On the other hand, there are M targets that move dynamically within a known road-network to accomplish complex tasks. More specifically, denote by $\mathcal{R} \triangleq (\mathcal{V}, \mathcal{E}, \eta)$ this road network, where $\mathcal{V} \subset \mathcal{W}$ is the set of nodes (called hubs) and $\mathcal{E} \subset \mathcal{V} \times \mathcal{V}$ is a set of straight roads connecting these hubs; and $\eta : \mathcal{E} \rightarrow \mathbb{R}^+$ measures the length of each road. On any road, each target $m \in \mathcal{M} \triangleq \{1, \dots, M\}$ follows the constant-velocity model [4] but with different velocities, i.e.,

$$\mathbf{y}_m(t+1) \triangleq G \mathbf{y}_m(t) + \mathbf{w}_m(t), \quad (2)$$

where $G \triangleq [\mathbf{I}_2, \Delta t \mathbf{I}_2; \mathbf{0}, \mathbf{I}_2]$ with sampling period $\Delta t > 0$; $\mathbf{y}_m(t) \in \mathcal{Y}$ is the target state including position and velocity at $t \geq 0$, and $\mathbf{w}_m(t) \sim \mathcal{G}(\mathbf{0}, W_m)$ is the Gaussian noise with zero mean and covariance W_m .

Moreover, some of these hubs are regions of interest that are labeled with semantic propositions, by a labeling function $\mathbf{L} : \hat{\mathcal{V}} \rightarrow 2^{AP}$, where AP is a set of propositions relevant to the tasks. For instance, the label “storage” indicates that certain goods are stored at this node. Note that the semantics are always known to both the targets and agents, but *time-varying*, e.g., the storage might relocate from one node to another, which can only be detected online. In addition, there is a high-level task for target m denoted by:

$$\varphi_m \in \varphi \triangleq \{\varphi_1, \dots, \varphi_J\}, \quad (3)$$

where each task within φ is expressed as LTL formulas over AP following the syntax described in the supplementary file due to limited space. Namely, as its intention, each target m is fulfilling one task within φ , although the exact task is unknown. Consequently, the route each target takes is also unknown. Nonetheless, the targets are assumed to be fully rational and follow the optimal strategy to accomplish the assigned task, e.g., [11], [22], [23].

C. Active Sensing Model

Moreover, each agent $n \in \mathcal{N}$ is equipped with a perception module that can: (i) recognize the semantic labels in AP . Namely, it is assumed that once the distance measure $\|\mathbf{x}_n(t) - \nu\| < D_n$ holds for a given range $D_n > 0$, the labels of hub $\nu \in \mathcal{V}$ are fully and correctly detected; and (ii) *actively* track the states of nearby targets via action $\mathbf{s}_n \subseteq \mathcal{M}$:

$$\mathbf{z}_{nm}(t) \triangleq h(\mathbf{x}_n(t), \mathbf{y}_m(t)) + \mathbf{v}(t), \quad \forall m \in \mathbf{s}_n(t); \quad (4)$$

where $h(\cdot)$ maps the current states of agent n and target m to the measurement $\mathbf{z}_{nm} \in \mathbb{R}^Z$, i.e., agent n w.r.t. target m ; $\mathbf{v}(t) \sim \mathcal{G}(\mathbf{0}, V(\mathbf{x}_n(t), \mathbf{y}_m(t)))$ is the Gaussian noise with a covariance $V(t)$ that varies by the agent and target states; and $\mathbf{s}_n(t)$ is the set of tracked targets by agent n at time t .

Denote by $n \in \overline{\mathcal{N}}(t) \subset \mathcal{N}$ if agent n is active at time $t \geq 0$. Then, its choice of actions $\mathbf{s}_n(t)$ is limited by:

$$\|\mathbf{x}_n(t) - \mathbf{y}_m(t)\| < R_n, \quad |\mathbf{s}_n(t)| \leq C_n; \quad (5)$$

where (I) $R_n > 0$ is a limited tracking range, since typical sensors such as radar and lidar are only valid locally; outside this range, the uncertainty $V(t)$ in (4) is set to infinity. Note that R_n may differ across agents. (II) there is a maximum number of targets each agent n can track simultaneously due to limited computation or energy resources, denoted by $C_n \geq 1$.

Remark 1. Note that the definition of action $\mathbf{s}_n$ is essential and differs from most existing work. Some methods [2], [4], [6], [7] assume that an agent can measure all targets at all time, while others [12], [13], [24] restrict each agent to only one target. In practice, detection and tracking incur energy or computation cost, so each agent must actively choose which nearby targets to measure, under a maximum number. ■

D. Target State Estimation and Tracking

Initially, each target $m \in \mathcal{M}$ has a prior Gaussian distribution for its initial state $\mathbf{y}_m(0) \sim \mathcal{G}(\mu_m^0, \Sigma_m^0)$, which is known to all agents. For time $t > 0$, each agent $n \in \mathcal{N}$ can move around and make observations of its assigned targets via (4). Let $\mathbf{Z}_m(t) \triangleq \{\mathbf{z}_{nm}(t) \mid m \in \mathbf{s}_n(t)\}_{n \in \mathcal{N}}$ denote the set of time-stamped observations for target m at time t . Due to the above sensing model and the coordination strategy discussed in the sequel, *at most one active agent* is assigned to monitor each target m . Consequently, $\mathbf{Z}_m(t)$ contains at most one local measurement. The state estimation is performed entirely locally by the assigned agent, and exchanged during task handovers. Given the locally accumulated measurements $\mathbf{Z}_m(0 : t)$, the posteriori distribution of target $m \in \mathcal{M}$ maintained by its assigned agent is denoted by:

$$\hat{\mathbf{y}}_m(t) \sim \mathcal{G}(\mu_m(t), \Sigma_m(t)), \quad (6)$$

which is the estimated mean and covariance of target m at time t , e.g., via the EKF [4], [25], when the target moves along one fixed road or at intersections as described in the sequel. The uncertainty may increase drastically if a target crosses an intersection without being monitored, but can be attenuated by predicting its high-level task φ_m and hence its

intended road. Accordingly, $\det(\Sigma_m(t))$ is used to evaluate the estimation quality of target m up to time $t \geq 0$.

Last but not least, a timer $\xi_\nu(t) \triangleq (\xi_{\nu,1}, \xi_{\nu,2})$ is added for some hubs as static targets of interest $\nu \in \hat{\mathcal{V}} \subset \mathcal{V}$, where $\xi_{\nu,1} \geq 0$ records the duration that it has been monitored; and $\xi_{\nu,2} \geq 0$ records the elapsed time since it is monitored last time. Thus, $\xi_{\nu,1}$ increases while the hub is monitored and resets when monitoring stops, whereas $\xi_{\nu,2}$ increases while the hub is not monitored and resets when monitoring resumes. To encourage the detection of updated semantic information, each hub should be monitored at least once every interval T_ν for a duration $d_\nu > 0$, i.e., let $\xi_\nu(t) \leq \hat{T}_\nu$ stand for

$$\xi_{\nu,1} \geq d_\nu, \xi_{\nu,2} \leq T_\nu, \forall \nu \in \hat{\mathcal{V}}; \quad (7)$$

where $\xi_{\nu,1}, \xi_{\nu,2}$ follow the dynamics described above.

E. Problem Statement

The overall problem is stated as a long-term constrained optimization over the motion and actions of all agents, i.e.,

$$\begin{aligned} \min_{\mathcal{N}(t), \{(\mathbf{u}_n, \mathbf{s}_n)\}} \quad & \left\{ \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^T \bar{N}(t) \right\} \\ \text{s.t.} \quad & \det(\Sigma_m(t)) < \gamma_m, \forall m, \forall t; \end{aligned} \quad (8a)$$

$$\xi_\nu(t) \leq \hat{T}_\nu, \forall \nu, \forall t; \quad (8b)$$

$$(1) - (6), \forall n, \forall m, \forall t; \quad (8c)$$

where $\bar{N}(t) > 0$ is the number of active agents in $\mathcal{N}(t)$ for $t \geq 0$; the objective is its long-run average; $\gamma_m > 0$ is a user-specified bound on the estimation uncertainty of each target $m \in \mathcal{M}$ in (8a); the visiting frequency and duration for static hubs in $\hat{\mathcal{V}}$ are bounded in (8b); and constraints in (8c) collect the dynamics, sensing, and estimation models.

Note that γ_m is a key design parameter. If it is close to the minimum variance achievable when one agent monitors one target, the required team size is M ; if it is sufficiently large, one agent may suffice by observing the targets in sequence. More numerical examples are provided in Sec. IV.

Remark 2. The determinant $\det(\Sigma_m(t))$ is utilized in (8a) and (24) rather than the trace or maximum eigenvalue $\lambda_{\max}(\Sigma_m(t))$. Theoretically, bounding $\det(\Sigma_m(t))$ directly restricts the Gaussian differential entropy, aligning with standard metrics in recent active tracking literature [2], [10], [20]. Computationally, while $\lambda_{\max}(\Sigma_m(t))$ penalizes worst-case directional uncertainty, its non-analytic form compromises the employed gradient-based SQP solver [26]. Conversely, $\det(\Sigma_m(t))$ maintains strict smoothness, permitting efficient analytic trace-polynomial gradient evaluations. ■

III. PROPOSED SOLUTION

As illustrated in Fig. 2, the proposed solution is described in this section. The first part is intention-aware trajectory prediction for dynamic targets, which includes automaton-based intention inference and an intention-aware EKF predictor in Sec. III-A. The second component in Sec. III-B is the fully distributed dynamic assignment of monitoring tasks, aimed at minimizing the number of active agents. The third component

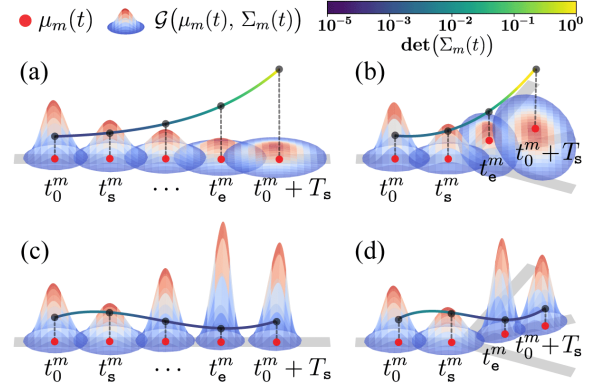


Fig. 3. Illustration of how the posterior uncertainty $\hat{\Sigma}_m$, derived from (9) and (10), differs from the prior uncertainty $\hat{\Sigma}_m$ in (9): (a) prior and (c) posterior on a road segment; (b) prior and (d) posterior at an intersection.

in Sec. III-C is ENMPC-based online navigation control for each agent, ensuring that the estimation uncertainty of any target remains bounded. Each component operates online and distributed with different triggering conditions and horizons.

A. Intention-aware Trajectory Prediction

As a fundamental prerequisite for the following components, this section presents how the future trajectories of all targets should be predicted, depending on whether the target is on a road segment or at an intersection.

1) *EKF-based Prediction on Road Segments:* Let the estimated state of target $m \in \mathcal{M}$ be $\hat{\mathbf{y}}_m(t_s) \triangleq \mathcal{G}(\hat{\mu}_m(t_s), \hat{\Sigma}_m(t_s))$ at time $t_s \geq 0$, where the mean belongs to one road segment of $\mathcal{R}$. Then, assuming that *no* measurements are obtained during the future period $t \in [t_s, t_e]$ for $t_e > t_s$, the predicted evolution of mean and covariance is given by:

$$\begin{aligned} \hat{\mu}_m(t+1) &= G \hat{\mu}_m(t), \\ \hat{\Sigma}_m(t+1) &= G \hat{\Sigma}_m(t) G + W_m; \end{aligned}$$

where G is the dynamic model of the target and W_m is the uncertainty from (2). On the other hand, assuming that agent n actively monitors target m during the period $[t_s, t_e]$, new measurements $\mathbf{z}_{nm}(t)$ are obtained for each $t \in [t_s, t_e]$. Thus, the posteriori estimation $\hat{\mathbf{y}}_m(t+1)$ is computed by the EKF [4], [27] via the standard prediction and update, i.e.,

$$\begin{aligned} \hat{\mu}_m(t+1) &= \mu'_m(t) + K_m(t)(\mathbf{z}_{nm}(t) - h(\mathbf{x}_n, \mu'_m(t))); \\ \hat{\Sigma}_m(t+1) &= \Sigma'_m(t) - K_m(t)H_m(t)\Sigma'_m(t); \end{aligned}$$

where $\mathbf{x}_n(t)$ is the agent state; $\mu'_m(t) = G \hat{\mu}_m(t)$; $H_m(t)$ is the linearized observation model of $h(\cdot)$ given $\mathbf{x}_n(t)$ and $\hat{\mu}_m(t)$; $\Sigma'_m(t)$ is the intermediate covariance computed by:

$$\begin{aligned} \Sigma'_m(t) &= G \hat{\Sigma}_m(t) G + W_m; \\ K_m(t) &= \Sigma'_m(t) H_m^T(t) R_m^{-1}(t); \\ R_m(t) &= H_m(t) \Sigma'_m(t) H_m^T(t) + V(\mathbf{x}_n(t), \hat{\mu}_m(t)); \\ H_m(t) &= \nabla_{\mathbf{y}} h(\mathbf{x}, \mathbf{y})|_{(\mathbf{x}_n(t), \mu'_m(t))}, \end{aligned}$$

and $V(\cdot)$ is the state-dependent uncertainty during observation from (4). The above estimation has been shown to be optimal [27] for the considered target model.

Therefore, depending on whether the target is monitored by an active agent, its state estimation is computed differently:

$$\begin{aligned}\hat{\mathbf{y}}_m(t+1) &\triangleq \rho_{\text{pri}}(\hat{\mathbf{y}}_m(t)), \\ \hat{\mathbf{y}}_m(t+1) &\triangleq \rho_{\text{pst}}(\hat{\mathbf{y}}_m(t), \mathbf{x}_n(t), \mathbf{z}_{nm}(t)),\end{aligned}\quad (9)$$

where the first provides the priori prediction; and the second is the posteriori prediction. Typically, the estimated uncertainty increases gradually without measurements, and decreases quickly once measurements are provided. Fig. 3 illustrates the difference between the prior and posterior uncertainty.

2) *Intention-aware Prediction at Intersections*: As discussed previously, since the targets move within a road network, the largest uncertainty in predicting their future trajectory occurs when they cross intersections. In particular, consider that the target is predicted to be at an intersection $\nu \in \mathcal{V}$ at time t with K outgoing directions. If no agents are monitoring it, the resulting estimation is given by:

$$\hat{\mathbf{y}}_m(t) \triangleq \mathcal{G}_{\text{apx}}\left(\{(w_m^k(t), \hat{\mathbf{y}}_m^k(t)), k \in \mathcal{K}\}\right), \quad (10)$$

where $\mathcal{G}_{\text{apx}}(\cdot)$ is the weighted mixture of a set of Gaussian distributions; $\mathcal{K} \triangleq \{1, \dots, K\}$; $\hat{\mathbf{y}}_m^k(t)$ is the posteriori estimation by (6) if target m follows the k -th direction at time $t-1$; $w_m^k(t) \in [0, 1]$ is the probability for target m taking the k -th direction and $\sum_{k \in \mathcal{K}} w_m^k(t) \triangleq 1$, $\forall m \in \mathcal{M}$; and $\|\hat{\mu}_m(t-1) - \nu\| < \varepsilon$, for one hub $\nu \in \mathcal{V}$ that $K = |\{\nu' | (\nu, \nu') \in \mathcal{E}\}|$ with a small margin $\varepsilon > 0$. As shown in Fig. 4, function $\mathcal{G}_{\text{apx}}(\cdot)$ is designed such that the uncertainty-aware Gaussian mixture augments the covariance with an intersection correction that raises $\det(\hat{\Sigma}_m(t))$ above $\bar{\gamma}_m$ when all weights are below ζ , and otherwise reduces to the standard mixture. Intuitively, $\mathcal{G}_{\text{apx}}(\cdot)$ effectively forces the coordination module to prioritize monitoring targets at intersections where their future paths are most unpredictable. The full definition of (10) is provided in the supplementary file.

Initially, without any prior knowledge about the task and behavior of the targets, the weights $\{w_m^k\}$ is unknown and initialized as uniform for all intersections, i.e., $w_m^k(t) \triangleq \frac{1}{K}$ for a K -intersection, $\forall k \in \mathcal{K}$. Thus, the resulting uncertainty in $\hat{\mathbf{y}}_m(t)$ would be larger than the given threshold γ_m in (8a). Consequently, any target that is at an intersection should be monitored by the agents. However, as motivated earlier, since the targets are assumed to be rational and undertake some high-level tasks, their future behavior including the routes can be predicted by observing their past behavior and inferring its intentions. Thus, predicting which task the target is performing would change the distribution of the weights $\{w_m^k\}$, thus reducing the estimation uncertainty for $\hat{\mathbf{y}}_m(t)$.

Definition 1 (Trajectory and Trace). The *trajectory* of target $m \in \mathcal{M}$ up to time $t > 0$ is defined as $\hat{\mathbf{Y}}_m(t) \triangleq \hat{\mathbf{y}}_m(0) \cdots \hat{\mathbf{y}}_m(t)$, as a sequence of estimated states. Moreover, its associated *trace* is defined as $\sigma_m(t) \triangleq \sigma_0 \cdots \sigma_L$, where $\sigma_\ell = \mathbf{L}(\nu_\ell)$ are the labels associated with the hub $\nu_\ell \in \mathcal{V}$ that target m has visited along its trajectory. ■

Definition 2 (Intention). The *intention* of target $m \in \mathcal{M}$ is defined as $\chi_m : \varphi \rightarrow [0, 1]$, where $\chi_m(\varphi_j) \in [0, 1]$ is the probability of target m performing task $\varphi_j \in \varphi$; and $\sum_{\varphi_j \in \varphi} \chi_m(\varphi_j) = 1$. ■

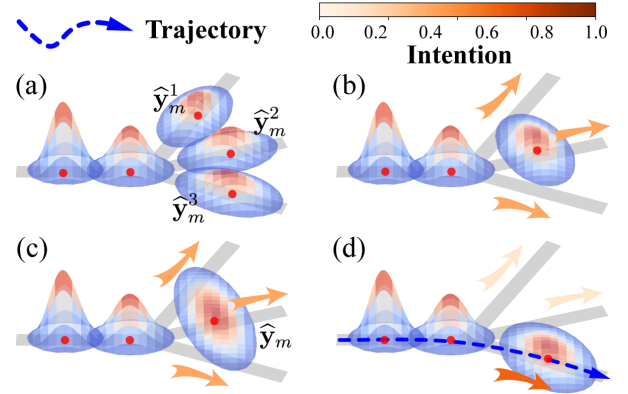


Fig. 4. Posterior uncertainty $\hat{\Sigma}_m$ via (10) at an intersection under different intention weights: (a) individual directional posteriors; (b) uniform Gaussian mixture; (c) intention-aware mixture with initial weights; and (d) intention-aware mixture after convergence.

Problem 1. Given the past trace $\sigma_m(t)$ and the current state estimation $\hat{\mathbf{y}}_m$ for target $m \in \mathcal{M}$, update: (I) the intention χ_m ; and (II) the weights $\{w_m^k(t)\}$ for the *next* intersection. ■

Since this problem is independent between targets, the subscript m above is mostly omitted for brevity in this part. The proposed method consists of two main steps: (I) the intention is updated based on how likely the past trace would result in an accepting run within the product automaton associated with each task; (II) the weights are updated via the Bayesian rule given the updated intention and the network structure.

To begin with, LTL specifications can be translated into an equivalent automaton, which can accept the same traces that satisfy the formula. Specifically, let $\mathcal{B}_j = (Q_j, Q_{j,0}, AP_j, \delta_j, Q_{j,F})$ denote the NBA associated with each $\varphi_j \in \varphi$, where the notations follow [28] and described in the supplementary file due to limited space. Furthermore, via the standard procedure [28], the Product Büchi Automaton (PBA) between the network $\mathcal{R}$ and $\mathcal{B}_j$ is given by $\mathcal{P}_j \triangleq \mathcal{R} \otimes \mathcal{B}_j = (Q'_j, \delta'_j, \eta'_j, Q'_{j,0}, Q'_{j,F})$, where $Q'_j = \mathcal{V} \times Q_j$; $\delta'_j : Q'_j \rightarrow 2^{Q'_j}$ such that $(\hat{v}, \hat{q}) \in \delta'_j((v, q))$ if and only if $(v, \hat{v}) \in \mathcal{E}$ and $\hat{q} \in \delta_j(q, \mathbf{L}(v))$; $\eta'_j : \delta'_j \rightarrow \mathbb{R}^+$ that $\eta'_j((v, q), (\hat{v}, \hat{q})) = \eta(v, \hat{v})$; $Q'_{j,0} = \mathcal{V} \times Q_{j,0}$; and $Q'_{j,F} = \mathcal{V} \times Q_{j,F}$. Note that $\mathcal{P}_j$ remains a Büchi automaton, i.e., its accepting path obeys the prefix-suffix structure.

Furthermore, denote by $\mathbb{P}(\sigma_{\ell+1} | \sigma_\ell, \varphi_j)$ the conditional probability of the next label generated by this target being $\sigma_{\ell+1} \subset AP$, given its task φ_j and past trace $\sigma_\ell \subset \sigma_m$. It should be proportional to how likely it can be generated by this target and how likely it can lead to the satisfaction of φ_j :

$$\mathbb{P}(\sigma_{\ell+1} | \sigma_\ell, \varphi_j) \propto \mathbb{P}(\sigma_{\ell+1} \in \{\mathbf{L}_m(\nu_{\ell+1})\}) \cdot \left(\min_{q'_\ell \in Q'_{j,\ell}} \text{dist}(q'_\ell, \sigma_{\ell+1}, Q'_{j,F}) \right)^{-1}, \quad (11)$$

where the first part is the probability that $\sigma_{\ell+1}$ can be generated by the neighboring hubs $\nu_{\ell+1} \in \{\nu \in \mathcal{R} | (\nu_\ell, \nu) \in \mathcal{E}\}$; $Q'_{j,\ell} \subset Q'_j$ are the reachable states in $\mathcal{P}_j$ given the past sequence σ_ℓ ; and $\text{dist}(q'_\ell, \sigma_{\ell+1}, Q'_{j,F})$ returns the minimum cost of all paths in $\mathcal{P}_j$ that (I) start from q'_ℓ and end in *any* accepting state in $Q'_{j,F}$; (II) have $\sigma_{\ell+1}$ as the first input word. Given $\sigma_{\ell+1}$, the associated hub $\nu_{\ell+1}$ is one neighbor of ν_ℓ .

Algorithm 1: Local Intention Update for Target m

Input: $\sigma_m(t)$, $\hat{\mathbf{Y}}_m(t)$, $\hat{\mathbf{y}}_m(t)$, $\mathcal{R}$, φ .

Output: $\sigma_m(t)$, $\hat{\mathbf{Y}}_m(t)$, χ_m , $\{w_m^k(t)\}$.

- 1 Compute $\mathcal{B}_j$ and $\mathcal{P}_j$ for each $\varphi_j \in \varphi$;
 - 2 **while** not terminated **do**
 - 3 Update $\hat{\mathbf{Y}}_m(t)$ given $\hat{\mathbf{y}}_m(t)$;
 - 4 Update $\sigma_m(t)$ given $\hat{\mathbf{Y}}_m(t)$;
 - 5 Update $Q'_{j,\ell}$ by (12);
 - 6 Compute $\mathbb{P}(\sigma_{\ell+1} | \sigma_\ell, \varphi_j)$ by (11);
 - 7 Compute $\mathbb{P}(\varphi_j | \sigma_{\ell+1})$ by (13);
 - 8 Update χ_ℓ by (14);
 - 9 Compute weights $\{w_m^k(t)\}$ by (15);
-

Thus, the set of reachable states $Q'_{j,\ell}$ is computed recursively online each time a new label is produced, i.e.,

$$Q'_{j,(\ell+1)} \triangleq \left\{ (\nu_{\ell+1}, q_{\ell+1}) | q_{\ell+1} \in \delta_j(q_\ell, \sigma_{\ell+1}), \forall q'_\ell \in Q'_{j,\ell} \right\}, \quad (12)$$

where $Q'_{j,\ell}$ is stored separately for each task $\varphi_j \in \varphi$. This allows the Bayesian module to penalize task hypotheses whose logical requirements are physically distant or unreachable given the actual movement of the target. Detailed algorithms are omitted here and refer to related work [22], [23], [29].

Consequently, the posterior likelihood of each target performing different tasks is computed via the Bayesian rule, i.e.,

$$\mathbb{P}(\varphi_j | \sigma_{\ell+1}) \propto \mathbb{P}(\sigma_{\ell+1} | \sigma_\ell, \varphi_j) \cdot \chi_\ell(\varphi_j), \quad (13)$$

which is then normalized across all $\varphi_j \in \varphi$; and $\chi_\ell(\varphi_j)$ is the estimated intention at the ℓ -th update. Thus, the intention prediction $\chi(\cdot)$ can be updated by a balanced average between the prior and posterior distribution, i.e.,

$$\chi_{\ell+1}(\varphi_j) \triangleq (1 - \varepsilon)\mathbb{P}(\varphi_j | \sigma_{\ell+1}) + \varepsilon\chi_\ell(\varphi_j), \quad (14)$$

where $\chi_{\ell+1}(\cdot)$ denotes the posterior prediction of the intention after the ℓ -th update; and $\varepsilon \in [0, 1]$ is a chosen parameter. Thus, the weight parameters $\{w_m^k\}$ can be updated as follows:

$$w_m^k \propto \sum_{\varphi_j \in \varphi} \mathbb{P}(\mathbf{L}(\nu_{\ell+1}^k) | \sigma_\ell, \varphi_j) \cdot \chi_{\ell+1}(\varphi_j), \quad (15)$$

where $(\nu_\ell, \nu_{\ell+1}^k) \in \mathcal{E}$ is the connecting hub of ν_ℓ if the k -th fork is taken; $\mathbf{L}(\nu_{\ell+1}^k)$ is the label of hub $\nu_{\ell+1}^k$ which follows (11) and the uncertainty of its label is set inversely proportional to $(T_{\nu_{\ell+1}^k} - \tau_{\nu_{\ell+1}^k})$ as the latest time it has been visited. Afterwards, all weights in $\{w_m^k\}$ are normalized and updated in (10). It is worth noting that the above procedure can be applied both when σ_ℓ is part of the plan prefix and the suffix. The above procedure is summarized in Alg. 1. An example of how $\{w_m^k\}$ is updated online given the past trajectory of a target is shown in Fig. 5.

Remark 3. Different from the proposed method of intention prediction in [30], this work utilizes the complete past trace as in (11), not just the current system state. This is essential as general LTL tasks, rather than simple reach-and-avoid tasks, make the past trace relevant to future behavior, also as the labeling function over the hubs is time-varying. ■

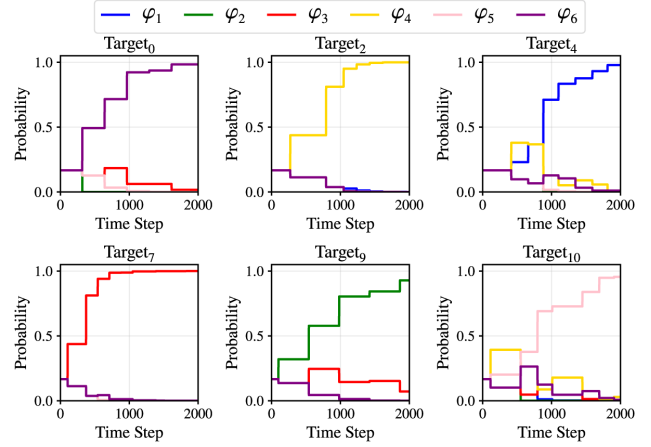


Fig. 5. Illustration of how χ_ℓ is updated online for 6 dynamic targets, given their past trajectories under 6 hypotheses of tasks (in different colors).

Remark 4. As emphasized earlier, the targets are assumed to act *rationally* by pursuing optimal paths within $\mathcal{R}$, e.g., [11], [22], [23]. Practical irrational behaviors such as random detours can flatten the Bayesian intention priors and degrade the prediction to a conservative Gaussian mixture at intersections. As shown in the sequel, this may require larger fleets but does not affect overall correctness, since the framework adapts online to such deviations. Jointly estimating target intention and rationality remains ongoing work. ■

Lemma 1. Let the road network $\mathcal{R}$ be static and known, and the actual task adopted by the target is φ_{j^*} . Assuming that the estimated past trace σ matches the actual trace of the target, the predicted intention $\chi_\ell(\cdot)$ via (14) satisfies that $\varphi_{j^*} = \underset{\varphi_j \in \varphi}{\operatorname{argmax}} \{\chi_\ell(\varphi_j)\}$ given σ_ℓ as $\ell \rightarrow \infty$.

Proof. Fix any competing task $\varphi_j \neq \varphi_{j^*}$, and denote by $p_{\ell,j} \triangleq \mathbb{P}(\sigma_{\ell+1} | \sigma_\ell, \varphi_j)$ the likelihood of the next observed label under task φ_j . Since the estimated trace σ_ℓ matches the actual trace and the target actually executes φ_{j^*} , the realized label sequence is always at least as compatible with φ_{j^*} as with any competing task under the update model in (11). Hence, $p_{\ell,j^*} \geq p_{\ell,j}$ for all $\ell \geq 0$, and the inequality is strict whenever φ_j disagrees with the realized continuation. Next, define the posterior ratio $\Lambda_\ell(j) \triangleq \mathbb{P}(\varphi_{j^*} | \sigma_\ell) / \mathbb{P}(\varphi_j | \sigma_\ell)$. By (13), the normalization constants cancel, so $\Lambda_{\ell+1}(j) = \Lambda_\ell(j) p_{\ell,j^*} / p_{\ell,j}$. Therefore, $\Lambda_\ell(j)$ is nondecreasing in ℓ , and it increases strictly whenever the competing task φ_j becomes less consistent with the realized trace than φ_{j^*} . Since the trace is generated by the true task φ_{j^*} , every competing task is penalized after sufficiently many informative updates. Consequently, $\Lambda_\ell(j) \rightarrow \infty$, which implies $\mathbb{P}(\varphi_{j^*} | \sigma_\ell) \rightarrow 1$ and $\mathbb{P}(\varphi_j | \sigma_\ell) \rightarrow 0$ for every $\varphi_j \neq \varphi_{j^*}$. Finally, (14) updates χ_ℓ by averaging the current posterior with the previous intention estimate. Thus, $\chi_\ell(\varphi_{j^*})$ inherits the same eventual dominance as the posterior sequence, whereas $\chi_\ell(\varphi_j) \rightarrow 0$ for all $\varphi_j \neq \varphi_{j^*}$. Hence, there exists $L > 0$ such that $\varphi_{j^*} = \underset{\varphi_j \in \varphi}{\operatorname{argmax}} \chi_\ell(\varphi_j)$ for all $\ell \geq L$. □

B. Distributed Assignment of Monitoring Tasks

Given the predicted trajectories of all targets from (9), it can be inferred that the uncertainty of some targets might exceed the given bound γ_m in (8a). In addition, the visiting interval for the hubs might exceed T_ν in (8b). Consequently, monitoring tasks are required for the targets and hubs. This section first formulates the assignment problem for monitoring tasks, followed by a distributed coordination algorithm to generate a stable assignment for the fleet, as shown in Fig. 6.

1) *Monitoring Task for Targets and Hubs:* To begin with, since the computation of $\hat{\mathbf{y}}_m(t)$ in (9) depends on the agent trajectory $\mathbf{x}_n(t)$, the following assumption is imposed:

Assumption 1. The initial uncertainty $\hat{\mathbf{y}}_m(t_0)$ at $t_0 \geq 0$ satisfies $\mathbf{det}(\hat{\Sigma}_m(t_0)) < \gamma_m$. Then, there always exists a choice of $\mathbf{x}_n(t)$ such that $\mathbf{det}(\hat{\Sigma}_m(t)) < \gamma_m$ via (9), $\forall t \geq t_0$. ■

In other words, the above assumption states that one agent is sufficient to monitor one target such that its uncertainty stays below the given bound. This assumption stems from the application that a few UAVs are deployed to monitor a large number of dynamic targets. Thus, cooperative tracking of a single target by multiple agents as in [4], [6], [7], [13], [31], [32] is not considered in this work. Due to the inherent combinatorial complexity of the assignment process, an approximation method is introduced here to bound the duration required for one agent to monitor one target without specifying the agent trajectory, such that the uncertainty remains below the threshold. The exact optimization of agent trajectories is performed in Sec. III-C afterwards. More specifically, assume that the target $m \in \mathcal{M}$ is monitored by agent $n \in \mathcal{N}$ during the monitoring period $[t_s^m, t_e^m]$, where $t_0 < t_s^m < t_e^m$. The starting time t_s^m is chosen between $[t_0, \bar{t}_s]$, where $\bar{t}_s$ is the latest starting time satisfying the following constraints:

$$\begin{aligned} \hat{\mathbf{y}}_m(t+1) &= \rho_{\text{pri}}(\hat{\mathbf{y}}_m(t)), \quad \forall t \in [t_0, \bar{t}_s]; \\ \mathbf{det}(\hat{\Sigma}_m(\bar{t}_s - 1)) &< \gamma_m, \quad \mathbf{det}(\hat{\Sigma}_m(\bar{t}_s)) \geq \gamma_m; \end{aligned} \quad (16)$$

which means the uncertainty grows above the threshold at $\bar{t}_s$ if the target is not monitored during $[t_0, \bar{t}_s - 1]$. Thus, given a choice of the starting time $t_s^m \in [t_0, \bar{t}_s]$, the ending time t_e^m is minimized to reduce the monitoring period such that the agent can potentially monitor more targets. Namely, t_e^m is chosen as the earliest instant such that the optimistic prediction of the target uncertainty is below another threshold $\bar{\gamma}_m < \gamma_m$, i.e.,

$$\begin{aligned} \hat{\mu}_m(t+1) &= G\hat{\mu}_m(t), \quad \forall t \in [t_s^m, t_e^m]; \\ \hat{\Sigma}_m(t+1) &= \min_{\{\mathbf{x}_n(t)\}} \{\Sigma'_m(t) - K_m(t)H_m(t)\Sigma'_m(t)\}; \\ \mathbf{det}(\hat{\Sigma}_m(t_e^m)) &< \bar{\gamma}_m; \end{aligned} \quad (17)$$

where the mean $\hat{\mu}_m(t)$ is updated based on (9); and the covariance $\hat{\Sigma}_m(t)$ is updated based on (9), assuming the perfect measurements $\{\mathbf{z}_{mn}\}$ according to the predicted trajectory at the best observation position $\mathbf{x}_n(t)$, during each step of the monitoring period (t_s^m, t_e^m) . This process continues until the predicted uncertainty is smaller than the given threshold $\bar{\gamma}_m$. In other words, the uncertainty is predicted to increase by (9) during $[t_0, t_s^m]$, and then decrease by (17) during $[t_s^m, t_e^m]$. If all conditions above hold, the choice of schedule (t_s^m, t_e^m) is

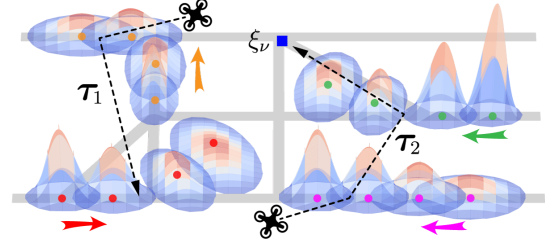


Fig. 6. Illustration of the distributed coordination scenario, where two neighboring agents communicate and exchange their local plans (dashed lines) and the prediction of all assigned targets (colored solid dots with uncertainty).

called *feasible*. These conditions are fundamental to determine the duration of monitoring task during online task assignment, and to predict the evolution of estimation uncertainty during trajectory optimization. Evolution of the uncertainty during the monitoring period is illustrated in Fig. 3.

Remark 5. Thresholds γ_m and $\bar{\gamma}_m$ in (16) and (17) are theoretically guided by the EKF steady-state covariance $\Sigma_m(t)$, which is determined by target process noise W_m and sensor noise $V(\cdot)$. Specifically, $\bar{\gamma}_m$ must strictly exceed $\mathbf{det}(\hat{\Sigma}_m)$ to ensure monitoring tasks conclude in finite time. The gap $(\gamma_m - \bar{\gamma}_m)$ dictates the trade-off between fleet efficiency and tracking precision. Larger margins relax the routing time windows, yielding greater flexibility to reduce the fleet size at the expense of reduced tracking accuracy. ■

Last but not least, for each hub of interest $\nu \in \hat{\mathcal{V}}$, its visiting schedule $[t_s^\nu, t_e^\nu]$ is defined similarly as the starting and ending time that $t_0 < t_s^\nu < t_e^\nu$. The schedule is called *feasible* if the following conditions hold:

$$t_e^\nu \geq t_s^\nu + d_\nu, \quad t_s^\nu \leq t_{e,-}^\nu + T_\nu; \quad (18)$$

where the first condition requires that the hub is visited for a minimum duration of $d_\nu > 0$; the second indicates that the hub is visited at least once every interval T_ν , with $t_{e,-}^\nu$ being the end of the previous visit. In other words, both the starting time and the duration should fulfill the above constraints.

2) *Formulation of Task Assignment Problem:* Thus, the monitoring plan for agent $n \in \mathcal{N}$ is defined as:

$$\tau_n \triangleq (m_1, t_s^{m_1}, t_e^{m_1}) \cdots (m_{L_n}, t_s^{m_{L_n}}, t_e^{m_{L_n}}), \quad (19)$$

where $(m_\ell, t_s^{m_\ell}, t_e^{m_\ell})$ is the schedule for monitoring targets and hubs $m_\ell \in \mathcal{M} \cup \hat{\mathcal{V}}$, $\forall \ell = 1, \dots, L_n$. With a slight abuse of notation, denote by $\mathbf{m}_n \triangleq m_1 m_2 \cdots m_{L_n}$ the sequence of assigned targets $\mathcal{M}_n \subseteq \mathcal{M}$ and hubs $\hat{\mathcal{V}}_n \subseteq \hat{\mathcal{V}}$ for agent n , and by $T_{\tau_n} \triangleq t_e^{m_{L_n}}$ the complete plan duration. For inactive agents $n \in \mathcal{N}_0$, its local plan τ_n is simply set to null. Thus, denoted by $\hat{\tau} \triangleq \{\tau_n, \forall n\}$ the complete plan of the fleet, of which the feasibility is defined as follows.

Definition 3 (Feasible Plan). A complete plan $\hat{\tau}$ is *feasible* if the contained schedules are feasible for all targets by (17) and for all hubs by (18). Otherwise, it is infeasible. ■

Moreover, a complete plan is associated with a *multi-objective* cost defined by a tuple:

$$C(\hat{\tau}) \triangleq (F_{\hat{\tau}}, N_{\hat{\tau}}, T_{\hat{\tau}}), \quad (20)$$

where the first element $F_{\hat{\tau}} \in \mathbb{N}$ is the number of infeasible local plans; the second element $N_{\hat{\tau}} \leq N$ is the number of active agents; and the last element $T_{\hat{\tau}} \triangleq \max_{n \in \mathcal{N}} \{T_{\tau_n}\} > 0$ is the maximum plan duration across all active agents, namely the overall *makespan*. Given this multi-objective cost, the optimality of two complete plans can be compared by the dominance relation as follows. Note that if any local plan τ_n is infeasible, both terms T_{τ_n} and $T_{\hat{\tau}}$ are set to ∞ .

Definition 4 (Plan Dominance). A complete plan $\hat{\tau}_1$ *dominates* another plan $\hat{\tau}_2$ if $C(\hat{\tau}_1) \leq C(\hat{\tau}_2)$, or simply τ_1 is better than τ_2 . ■

Note that “ $\leq$ ” is the generalized inequality for vectors [33], i.e., $(f_1, n_1, t_1) \leq (f_2, n_2, t_2)$ if $f_1 \leq f_2$, $n_1 \leq n_2$ and $t_1 \leq t_2$ hold; however two vectors can not be equal in the above definition. In other words, a complete plan with a smaller number of infeasible local plans, active agents and overall duration is preferred. A centralized solution can be formulated as a mixed integer linear program (MILP) with $3 \cdot M \cdot N$ variables in $\hat{\tau}$, or as a multi-vehicle routing problem with time windows (MVRP-TW) [21] as solved by existing solvers [34]. However, it introduces latency due to information gathering and central computation. Since targets move dynamically, the solution becomes suboptimal or infeasible, requiring constant re-computation. As shown in Fig. 6, a distributed assignment algorithm is preferred for online plan generation under distributed communication and local computation.

Definition 5 (Local Operations). The agents can perform *three* types of local operations to change the overall plan $\hat{\tau}$:

(I) *Shuffle* operation. Any active agent $n \in \mathcal{N}$ can change its own local plan τ_n via a shuffle operation by changing the sequence of targets (or hubs) and their schedules in (19);

(II) *Give* operation. Agent $i \in \mathcal{N}$ performs a give operation with agent $j \in \mathcal{N}$ by removing one or more targets (or hubs) from τ_i and adding it to τ_j .

(III) *Swap* operation. Agent $i \in \mathcal{N}$ performs a swap operation with agent $j \in \mathcal{N}$ by swapping exactly one target (or hub) within τ_i with one target (or hub) within τ_j . ■

Note that only an active agent can perform a shuffle operation, while the give or swap operation can happen not only between two active agents, but also between one active agent and one inactive agent. However, the give operation is directional, i.e., moving tasks from one agent to another. The exact algorithmic details are described in the sequel.

Definition 6 (Stable Plan). A complete plan $\hat{\tau}^*$ is called *stable* w.r.t. (20) only if: (I) all local plans are feasible, i.e., $F_{\hat{\tau}^*} = 0$; (II) all targets are assigned, i.e., there exists $n_m \in \mathcal{N}$ such that $m \in \tau_{n_m}$, $\forall m \in \mathcal{M}$; (III) any agent $n \in \mathcal{N}$ can not generate a better plan than $\hat{\tau}^*$ via a shuffle operation; (IV) any two agents $i, j \in \mathcal{N}$ can not generate a better plan than $\hat{\tau}^*$ via either a give or swap operation. ■

Assumption 2. Given the prediction $\{\hat{y}_m(t_0), \forall m \in \mathcal{M}\}$ for all targets and the intervals $\{\xi_\nu(t_0), \forall \nu \in \hat{\mathcal{V}}\}$ at $t_0 \geq 0$ for all hubs, the assignment problem is feasible if there exists a feasible plan $\hat{\tau}$ by assigning each target and hub to exactly one agent, i.e., with $|\mathcal{M}| + |\hat{\mathcal{V}}|$ agents. ■

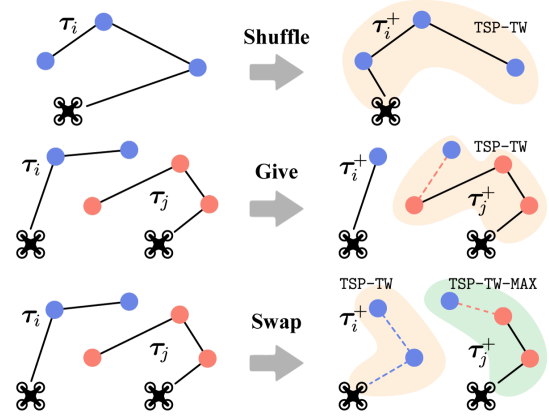


Fig. 7. Illustration of the local task coordination algorithm between two agents i, j that satisfy the conditions in Lemma 2. Three operations can be performed to minimize the overall cost in (23).

Remark 6. Assump. 2 serves as a practical safe initialization protocol, where a fleet with redundant agents is deployed to ensure a conservative one-to-one coverage at t_0 . This prevents the mission from being inherently ill posed. From this safe baseline, the proposed coordination scheme reduces the number of active agents such that redundant agents become inactive by hovering or parking. ■

Problem 2. Assuming that the assignment problem is feasible at t_0 by Assump. 2, design a distributed coordination strategy for the agents to compute a stable plan $\hat{\tau}^*$. ■

3) *Distributed Task Coordination*: This section presents the distributed coordination algorithm for computing a stable plan from local communication alone. First, give or swap operations need only be considered between nearby agents.

Lemma 2. A give or swap operation from agent $i \in \mathcal{N}$ to agent $j \in \mathcal{N}$ under a feasible plan $\hat{\tau}$ can not generate a better plan if the following conditions hold for all $m \in \mathcal{M}_i$: (I) $\min_{t' \in [t, \bar{t}_s^m]} \{T(\mathbf{x}_j(t'), \mu_m(t'))\} + t_0 > \bar{t}_s^m$; and (II) $\min_{t_s^m \in [t, \bar{t}_s^m]} \{t_e^m - t_s^m\} + T_{\tau_j} > T_{\hat{\tau}}$.

Proof. Let $\hat{\tau}^+$ and τ_j^+ denote the updated complete plan and local plan for agent j after receiving target m , respectively. If condition (I) holds, the earliest possible arrival time t_{arr} of agent j at target m satisfies the condition below: $t_{\text{arr}} \triangleq t_0 + \min_{t' \in [t, \bar{t}_s^m]} T(\mathbf{x}_j(t'), \mu_m(t')) > \bar{t}_s^m$, which implies that the actual monitoring start time t_s^m must satisfy $t_s^m \geq t_{\text{arr}}$. This strict inequality yields $t_s^m > \bar{t}_s^m$, which violates the time window feasibility constraint. Consequently, the infeasibility yields $F_{\hat{\tau}^+} \geq 1 > F_{\hat{\tau}} = 0$, rendering $\hat{\tau}^+$ strictly dominated. If condition (II) holds, assume τ_j^+ remains feasible, i.e., $F_{\hat{\tau}^+} = 0$. By the triangle inequality of the TSP-TW routing cost, the updated makespan of agent j is lower-bounded by its original makespan plus the minimum required service duration $\Delta T_m \triangleq t_e^m - t_s^m$ for the newly added target m , i.e., $T_{\tau_j^+} \geq T_{\tau_j} + \min_{t_s^m \in [t, \bar{t}_s^m]} \Delta T_m$. Substituting condition (II) into the above inequality yields $T_{\tau_j^+} > T_{\hat{\tau}}$. The new global makespan $T_{\hat{\tau}^+}$ satisfies $T_{\hat{\tau}^+} = \max_{n \in \mathcal{N}} \{T_{\tau_n^+}\} \geq T_{\tau_j^+} > T_{\hat{\tau}}$. Given the cost definition $C(\hat{\tau}) = (F_{\hat{\tau}}, N_{\hat{\tau}}, T_{\hat{\tau}})$ in (20), the operation strictly increases the global makespan without decreasing feasibility,

i.e., $C(\hat{\tau}^+) > C(\hat{\tau})$. Thus, the operation cannot generate a dominant plan, completing the proof. $\square$

Initially at $t_0 \geq 0$, each agent $i \in \mathcal{N}$ is assigned optimally to one target or hub $n \in \{1, \dots, (|\mathcal{M}| + |\hat{\mathcal{V}}|)\}$, given their initial position and predictions, e.g., via the unbalanced linear bottleneck assignment algorithm (LBAP) [35], [36]. By Assump. 2, this initial plan is feasible. However, to reduce the number of active agents, the agents must coordinate their local plans online for $t > t_0$. More specifically, due to the conditions in Lemma 2, each agent $i \in \mathcal{N}$ *only needs to* communicate with another agent j within its close vicinity, i.e.,

$$\mathbf{x}_j(t) \in \bigcup_{m \in \mathbf{m}_i} \left\{ \max_{t' \in [t, \bar{t}_s^m]} \{ \mathcal{O}(\mu_m(t'), (\bar{t}_s^m - t')v_{\max}) \} \right\}, \quad (21)$$

where $\mathcal{O}(c, r) \subset \mathbb{R}^2$ measures a circular area centered at $c \in \mathbb{R}^2$ with radius $r > 0$; and $v_{\max} > 0$ is the maximum velocity of all agents. This area can be computed by agent i *alone* from the predicted target trajectories. Since most agents lie outside it, the communication burden is reduced significantly. As shown in Fig. 7, if condition (21) holds, agents i, j follow the local coordination rules below:

(I) Agents i, j exchange their local plans and compare their durations. They proceed only if the following condition holds:

$$(T_{\tau_i} = \infty, T_{\tau_j} < \infty); \text{ or } (T_{\tau_i} < T_{\tau_j} < \infty); \quad (22)$$

$$\text{or } (|\tau_i| < |\tau_j|, (T_{\tau_i}, T_{\tau_j} < \infty)),$$

which terminates the local communication if not true.

(II) For the *give* operation, agent i scans its local targets or hubs $m \in \mathbf{m}_i$ and checks the second condition of Lemma 2. The admissible candidates are collected in $\mathbf{m}_{ij} \subseteq \mathbf{m}_i$. If $\mathbf{m}_{ij}$ is empty, the communication terminates. For each $m \in \mathbf{m}_{ij}$, agent i sends its index m and predicted trajectory $\hat{\mathbf{y}}_m$ to agent j . Agents i and j then solve local TSP-TW instances independently over $\mathbf{m}_i^+ \triangleq \mathbf{m}_i \setminus \{m\}$ and $\mathbf{m}_j^+ \triangleq \mathbf{m}_j \cup \{m\}$, yielding updated plans τ_i^+ and τ_j^+ . If the problem for agent j is infeasible, the makespan of the updated plan is set to infinity. Denote the results by $(\tau_i^+, \tau_j^+) = \text{Give}(\tau_i, \tau_j, m)$. The exact TSP-TW formulation is provided in the supplementary file. Consequently, let $\tau_{ij}^+ \triangleq \{(\tau_i^+, \tau_j^+)_m : m \in \mathbf{m}_{ij}\}$ denote the candidate updates obtained by giving target or hub m from agent i to agent j , while all other agents remain unchanged. Each candidate is evaluated by the following 5-tuple:

$$\left(\mathbf{1}_{\text{Infs}(\tau_j^+)}, \mathbf{1}_{\text{Infs}(\tau_i^+)}, |\tau_i^+|, T_{\tau_i^+}, \max\{T_{\tau_j^+}, T_{\hat{\tau}}\} \right), \quad (23)$$

where $\text{Infs}(\cdot)$ tests infeasibility. The pair $(\tau_i^{+,*}, \tau_j^{+,*}) \in \tau_{ij}^+$ is the minimizer under the dominance relation in Def. 4. If all pairs satisfy $T_{\tau_j^+} = \infty$, the procedure follows the *swap* step.

(III) For the *swap* operation, agent i enumerates each target $m \in \mathbf{m}_i$ and sends its predicted trajectory to agent j , which forms $\mathbf{m}_j^+ = \mathbf{m}_j \cup \{m\}$ and solves a relaxed routing problem that may drop at most one element to recover feasibility, $(\tau_j^+, m') \triangleq \text{TSP-TW-MAX}(\tau_j, \mathbf{m}_j^+)$. Agent i then updates its plan by removing m and admitting m' , $\tau_i^+ = \text{TSP-TW}(\tau_i, (\mathbf{m}_i \setminus \{m\}) \cup \{m'\})$. Denote this update by $(\tau_i^+, \tau_j^+, m') \triangleq \text{Swap}(\tau_i, \tau_j, m)$. Different from the *give* operation, this step may increase T_{τ_i} . Thus, let $\tau_{ij}^+ \triangleq \{(\tau_i^+, \tau_j^+)_{(m, m')} :$

Algorithm 2: Distributed Task Coordination

Input: $\tau_i, \tau_j, \{\hat{\mathbf{y}}_m, m \in \mathbf{m}_i \cup \mathbf{m}_j\}$.
Output: τ_i^+, τ_j^+ .

```

1 if Agents  $i, j$  satisfy (21) then
2   Exchange  $\tau_i, \tau_j$  and  $\{\hat{\mathbf{y}}_m\}$ ;
3   Add  $(\tau_i, \tau_j)$  to  $\tau_{ij}^+$ ;
4   if condition in (22) holds then
5     Agent  $i$  computes  $\mathbf{m}_{ij}$ ;
6     forall  $m \in \mathbf{m}_{ij}$  do
7        $(\tau_i^+, \tau_j^+) = \text{Give}(\tau_i, \tau_j, m)$ ;
8       Add  $(\tau_i^+, \tau_j^+)_m$  to  $\tau_{ij}^+$ ;
9     if  $\tau_{ij}^+$  not empty then
10      Find  $(\tau_i^{+,*}, \tau_j^{+,*}) \in \tau_{ij}^+$  by (23);
11    else
12      forall  $m \in \mathbf{m}_i$  do
13         $(\tau_i^+, \tau_j^+, m') = \text{Swap}(\tau_i, \tau_j, m)$ ;
14        Add  $(\tau_i^+, \tau_j^+)_{(m, m')}$  to  $\tau_{ij}^+$ ;
15      Find  $(\tau_i^{+,*}, \tau_j^{+,*}) \in \tau_{ij}^+$  by (23);
16  Update  $\tau_i^+ = \text{Shuffle}(\tau_i)$ ;
17  Confirm and update  $\tau_i^+, \tau_j^+$ ;
```

$m \in \mathbf{m}_{ij}\}$ denote the candidate plans generated by swapping target m of agent i with the resulting target m' of agent j . Each candidate is evaluated using the same 5-tuple in (23), and the minimizer is selected under the same dominance relation. Independently, any agent may perform a *shuffle* update to refine the local ordering and timing via $\tau_i^+ = \text{TSP-TW}(\tau_i, \mathbf{m}_i)$. The overall procedure is summarized in Alg. 2.

Theorem 3. *Under Alg. 2 and a static, undirected, and connected communication graph $\mathcal{G}_c$, the overall plan $\hat{\tau}^*$ converges in finitely many inter-agent communications to a Nash-stable plan in the sense of Def. 6.*

Proof. Let $C(\hat{\tau}^{(k)}) \triangleq (F_{\hat{\tau}}^{(k)}, N_{\hat{\tau}}^{(k)}, T_{\hat{\tau}}^{(k)})$ with the entries being the feasibility cost, the number of active agents, and the makespan at around k . By Alg. 2, an update is accepted only if $C(\hat{\tau}^{(k+1)}) < C(\hat{\tau}^{(k)})$, where the comparison follows the same dominance rule used in the local coordination step. By Assump. 2, the initial plan is feasible, and thus $F_{\hat{\tau}}^{(0)} = 0$. Since feasibility has the highest priority in the cost, every accepted update preserves $F_{\hat{\tau}}^{(k)} = 0$. Moreover, as Alg. 2 only shuffles, gives, or swaps previously assigned tasks, the overall assignment remains complete. For each fixed set of local tasks, the shuffle step solves the local TSP-TW problem and therefore enforces the local optimality required by Def. 6. For two-agent coordination, the accepted give or swap move is selected as the best candidate from Alg. 2. Hence, once Alg. 2 terminates, no single agent can improve its plan by shuffle and no admissible pair can improve the overall plan by give or swap. Finally, given the finite number of tasks, the number of feasible assignments is finite. Since each accepted update strictly decreases the cost and the first two entries of $C(\hat{\tau}^{(k)})$ are integer-valued, the sequence of accepted plans cannot be infinite. Thus, the algorithm terminates after finite rounds, and the terminal plan satisfies all conditions in Def. 6, thus being Nash-stable. This completes the proof. $\square$

Remark 7. The convergence result above is decoupled from the accuracy of trajectory prediction by time-scale separation. During each run of Alg. 2, the predicted target states are fixed and the distributed coordination evolves over discrete communication rounds. Before the intention inference converges, inaccurate predictions may enlarge the estimated uncertainty near intersections, yielding a conservative assignment that activates more agents to satisfy the uncertainty bounds. ■

C. Event-triggered and Informative Trajectory Optimization via Distributed NMPC

Given the schedule $\{\tau_n\}$, each active agent $n \in \bar{\mathcal{N}}$ should optimize its trajectory to monitor the assigned targets and hubs and ensure the uncertainty constraints, as shown in Fig. 8. Different from existing work with discretized actions [2], [4], [6], the main challenges are both the agent dynamics and uncertainty bounds are nonlinear, while the target behavior is observed online. Thus, the framework of distributed nonlinear model predictive control (NMPC) from [37], [38] is adopted.

Problem 3. Given the schedule τ_n of an active agent $n \in \bar{\mathcal{N}}$, compute the optimal sequence of control and monitoring action $(\mathbf{u}_n(t), \mathbf{s}_n(t))$, such that: (I) the uncertainties of all assigned targets $\mathcal{M}_n$ are bounded by (8a); (II) the monitoring intervals for assigned hubs $\hat{\mathcal{V}}_n$ are bounded by (18); and (III) the control cost as defined in the sequel is minimized. ■

To begin with, another optimization horizon $T_c > 0$ is introduced for the local trajectory optimization, which can be different across the agents. Given the past estimation of all targets $\{\hat{\mathbf{Y}}_m, \forall m \in \mathcal{M}_n\}$ at time t_0 and the monitoring intervals for all hubs $\{\xi_\nu, \forall \nu \in \hat{\mathcal{V}}_n\}$, the NMPC problem is formulated for each active agent $n \in \bar{\mathcal{N}}$ as follows:

$$\begin{aligned} & \min_{(\mathbf{u}_n(t), \mathbf{s}_n(t))} \sum_{t \in \mathcal{T}_c} J(\mathbf{x}_n(t), \mathbf{u}_n(t)) \\ & \text{s.t. } \mathbf{x}_n(t+1) \triangleq f(\mathbf{x}_n(t), \mathbf{u}_n(t)); \\ & \tilde{\mathbf{y}}_m(t+1) = \begin{cases} \rho_{\text{pri}}(\tilde{\mathbf{y}}_m(t)), & \forall m \notin \mathbf{s}_n(t), \\ \rho_{\text{pst}}(\tilde{\mathbf{y}}_m(t), \mathbf{x}_n(t)), & \forall m \in \mathbf{s}_n(t); \end{cases} \quad (24) \\ & |\mathbf{s}_n(t)| < C_n; \det(\tilde{\Sigma}_m(t)) < \gamma_m, \forall m \in \mathcal{M}_n; \\ & \xi_\nu(t) \leq \hat{T}_\nu, \forall \nu \in \hat{\mathcal{V}}_n; \\ & \mathbf{x}_n(t) \in \mathcal{W}, \mathbf{u}_n(t) \in \mathcal{U}; \end{aligned}$$

where each constraint above should hold for all $t \in \mathcal{T}_c \triangleq \{t_0, \dots, t_0 + T_c\}$; $(\mathbf{u}_n(t), \mathbf{s}_n(t))$ is the agent plan including control and action to be optimized; $\tilde{\mathbf{y}}_m$ is the *predicted* state estimation of each target $m \in \mathcal{M}_n$ if the agent follows the planned trajectory; the objective $J: \mathcal{W} \times \mathcal{U} \rightarrow \mathbb{R}^+$ is a general function to measure the control cost and smoothness, e.g.,

$$J(\mathbf{x}_n(t), \mathbf{u}_n(t)) \triangleq \|\mathbf{u}_n(t)\| + \beta \|\mathbf{x}_n(t) - \mathbf{x}_n(t-1)\|,$$

with $\beta \geq 0$ being the weighting parameter; the uncertainty measure $\hat{\Sigma}_m(t)$ is the priori and posterior estimation from (9) depending on whether target m is being monitored by agent n at time $t \in \mathcal{T}_c$; and the time intervals for assigned hubs in $\hat{\mathcal{V}}_n$

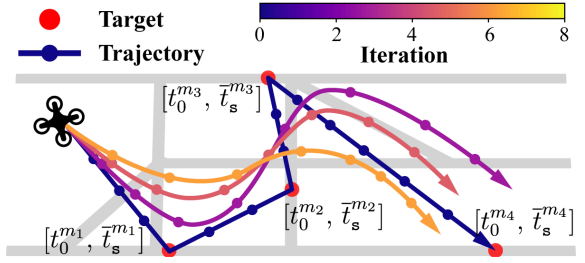


Fig. 8. Illustration of the dynamic trajectory optimization for each active agent, given its local schedule τ_n over assigned targets. Iterative SQP is applied to solve the NMPC problem in (24), with intermediate trajectories.

should be bounded. Denote by $\mathbf{m}_n \triangleq \mathcal{M}_n \cup \hat{\mathcal{V}}_n$ the set of targets and hubs within the local plan τ_n . Note that $\tilde{\mathbf{y}}_m$ is updated in the same way as $\hat{\mathbf{y}}_m$, following the priori and posteriori update in (9). Moreover, when target m is close to intersections, the posterior $\tilde{\mathbf{y}}_m$ is updated via (10).

It is worth noting that the above constrained optimization is different from the standard NMPC in [37] due to the variable $\mathbf{s}_n(t)$, as the monitoring action to decide which targets and hubs to monitor. It is discrete thus yielding a combinatorial complexity over the total number of targets and the planning horizon. Instead, a hierarchical structure is followed to decouple the optimization over $\mathbf{s}_n(t)$ and $\mathbf{u}_n(t)$. Moreover, the complexity is further reduced by adopting the technique of sequential quadratic programming (SQP) and the event-triggered update, as shown in Fig. 8.

1) *Monitoring Action:* Given the current state of agent n and all targets in $\mathbf{m}_n$ within its local plan τ_n at time t , the first C_n targets or hubs in $\mathbf{m}_n$ with the *smallest* margin to the respective allowed bounds at time t_0 , i.e.,

$$\mathbf{s}_n(t) = \underset{m, \nu \in \mathbf{m}_n}{\text{argmax_k}} \left\{ \frac{\det(\hat{\Sigma}_m(t))}{\gamma_m}, \frac{\xi_\nu(t_0)}{T_\nu} \right\}, \forall t \in \mathcal{T}_c; \quad (25)$$

where $\text{argmax_k}(\cdot)$ is the operator for choosing the k maximum elements in a set with $k \triangleq C_n$, i.e., as the maximum capacity of agent n . In other words, the targets with large estimation uncertainty and hubs with small time window of monitoring left are prioritized. Thus, once $\mathbf{s}_n$ is determined, it is kept constant during the whole horizon in the optimization (24). Note that if the total number of targets and hubs to be monitored is less than the capacity that $|\mathbf{m}_n| \leq C_n$, the above step to determine the monitoring action can be skipped as all targets and hubs can be monitored simultaneously, as long as their relative distances satisfy (5).

2) *Sequential Quadratic Programming (SQP):* Once $\mathbf{s}_n(t)$ is fixed, problem (24) becomes a nonlinear program over the state and input trajectories $\mathbf{X}_n, \mathbf{U}_n$ for $t \in \mathcal{T}_c$, and the posterior covariances $\hat{\Sigma}_m(t)$ for $m \in \mathbf{s}_n$. The determinant constraint is handled via an analytic trace-polynomial expression for $\det(\hat{\Sigma}_m(t))$, see [39] and the supplementary material for details. Denote by $\mathbf{Z}_n \triangleq [\mathbf{X}_n^T, \mathbf{U}_n^T]$. During real-time execution, the sequential quadratic programming is adopted, where the NMPC is written with the equality constraints $P(\mathbf{Z}_n) = 0$ capturing the agent dynamics and EKF updates, and the inequality constraints $Q(\mathbf{Z}_n) \leq 0$ enforcing the uncertainty bounds. Via analyzing the associated Lagrangian $\mathcal{L}(\mathbf{Z}_n)$, each

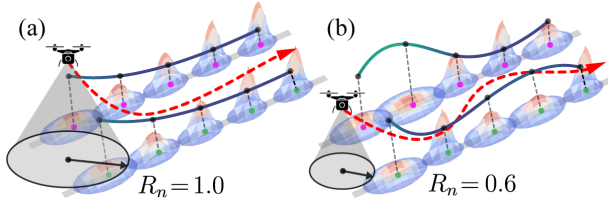


Fig. 9. Illustration of how the observation range may affect the optimal trajectory. **Left:** the agent can observe two targets simultaneously following the trajectory in the middle; **Right:** the agent can only observe one target at each time, yielding a wave motion to switch between the targets.

Newton iteration solves the following QP, i.e.,

$$\begin{aligned} \min_{\tilde{\mathbf{Z}}_n} & \nabla_{\mathbf{Z}_n} \bar{J}(\mathbf{Z}_n^k)^\top \tilde{\mathbf{Z}}_n + \tilde{\mathbf{Z}}_n^\top H_k \tilde{\mathbf{Z}}_n \\ \text{s.t.} & P(\mathbf{Z}_n^k) + \nabla_{\mathbf{Z}_n} P(\mathbf{Z}_n^k)^\top \tilde{\mathbf{Z}}_n = 0, \\ & Q(\mathbf{Z}_n^k) + \nabla_{\mathbf{Z}_n} Q(\mathbf{Z}_n^k)^\top \tilde{\mathbf{Z}}_n \leq 0, \end{aligned} \quad (26)$$

where $\bar{J}(\mathbf{Z}_n^k)$ is the total cost in (24); $\tilde{\mathbf{Z}}_n \triangleq \mathbf{Z}_n - \mathbf{Z}_n^k$ are the decision variables as the update relative to the k -th iteration $\mathbf{Z}_n^k$; and $H_k = \nabla_{\mathbf{Z}_n}^2 \mathcal{L}(\mathbf{Z}_n^k)$. Solutions update $\mathbf{Z}_n^k$ and the Lagrangian multipliers until convergence, yielding the desired control sequence $\mathbf{u}_n(t)$ over the horizon for each active agent $n \in \bar{\mathcal{N}}$. An example of the optimized trajectories under different observation ranges is shown in Fig. 9. Analytical derivations are provided in the supplementary material.

3) *Event-triggered Update:* Lastly, since the predicted motion of the targets are rather accurate along the road segments, the optimized agent trajectory $\mathbf{x}_n(t)$ remains valid for an extended period of time. Thus, instead of reformulating and resolving the problem in (24) at every time step and applying only the first control as the convention [37], [38], an event-triggered update law is proposed such that the optimized trajectory from (26) is tracked until at least one of the following conditions is met, i.e.,

$$\max_{m \in \mathbf{s}_n(t)} \{\text{Wass}(\hat{\mathbf{y}}_m(t), \tilde{\mathbf{y}}_m(t))\} > \delta_w; \quad (27a)$$

$$\det(\hat{\Sigma}_m(t)) < \bar{\gamma}_m, m \in \mathbf{s}_n(t); \quad (27b)$$

$$\xi_{\nu,1}(t) > d_\nu, \nu \in \mathbf{s}_n(t); \quad (27c)$$

where $\mathbf{s}_n(t)$ is derived in (25) which is unchanged within the horizon $\mathcal{T}_c$; $\text{Wass}(\cdot)$ in (27a) is the Wasserstein-2 distance between two Gaussian distributions [40]; $\hat{\mathbf{y}}_m(t)$ is the actual estimation based on online measurements via (9), while $\tilde{\mathbf{y}}_m(t)$ is the predicted estimation by the NMPC in (24); $\delta_w > 0$ is a given bound on the maximum distance between the predicted and actual estimations over all monitored targets. In this case, the actual estimation of some assigned targets deviates from the prediction during NMPC. This metric is chosen as the two predictions $\hat{\mathbf{y}}_m, \tilde{\mathbf{y}}_m$ are related. Moreover, the condition in (27b) checks if the uncertainty of any target is below the given bound $\bar{\gamma}_m$ as in (17), while the condition in (27c) checks if the monitoring duration of a hub has reached the bound d_ν in (8b). In this two cases, the monitoring tasks of some assigned targets and hubs are temporarily accomplished, meaning that the action $\mathbf{s}_n$ should be updated.

Thus, the planned trajectory $\mathbf{x}_n(t)$ of agent n is followed without replanning, until the actual estimation of certain targets deviates from the prediction; or the monitoring tasks

Algorithm 3: Distributed Fleet Monitoring

Input: $\mathcal{M}, \mathcal{R}, \{\hat{\mathbf{y}}_m(0)\}, \{\xi_\nu(0)\}, \{\bar{\gamma}_m\}, T_s, T_c$.

Output: $\bar{\mathcal{N}}(t), \{\hat{\mathbf{y}}_m(t)\}, \{\xi_\nu(t)\}, \{(\mathbf{u}_n, \mathbf{s}_n)\}$.

```

1 Initialize  $\{\tau_n\}$  via Alg. 2 at  $t = 0$ ;
2 while not terminated do
3   forall  $n \in \bar{\mathcal{N}}$  do
4     /* Motion Planning */
5     if (27) holds then
6       Form NMPC (24) given  $\tau_n, \{\hat{\mathbf{y}}_m\}, \{\xi_\nu\}$ ;
7       Solve hybrid plan  $\{(\mathbf{u}_n, \mathbf{s}_n)\}$ ;
8     /* Execution and Observation */
9     if  $\{(\mathbf{u}_n, \mathbf{s}_n)\}$  exists then
10      Apply  $(\mathbf{u}_n(t), \mathbf{s}_n(t))$ ;
11      Obtain measurements  $\{\mathbf{z}_{nm}\}$ ;
12      Update  $\{\hat{\mathbf{y}}_m, \forall m \in \mathcal{M}_n\}$  by (9);
13      Update  $\{\xi_\nu, \forall \nu \in \hat{\mathcal{V}}_n\}$ ;
14     /* Task Coordination */
15     if one task done or being requested then
16       Update local plan  $\tau_n$  by shuffle;
17       Update local data  $\Omega_n$  in (28);
18       Identify neighbors  $m$  by (2);
19       Local coordination via Alg. 2;
20       Update local plan  $\tau_n, \tau_m$ ;
21   Update  $\bar{\mathcal{N}}; t \leftarrow t + 1$ ;

```

of some assigned targets and hubs are accomplished. By then, the action $\mathbf{s}_n$ is updated by (25) given the latest estimation $\{\hat{\mathbf{y}}_m, m \in \mathcal{M}_n\}$ for targets and stamps $\{\xi_\nu, \nu \in \hat{\mathcal{V}}_n\}$ for hubs. Furthermore, the resulting NMPC is re-formulated and solved via the SQP proposed in (26), from which the solution $\{\mathbf{u}_n(t)\}$ is followed until the condition in (27) is met again. Consequently, the above procedure of distributed and event-triggered NMPC (ENMPC) can be executed indefinitely even if the local plan τ_n does not change. However, it can happen that (24) does not have a feasible solution as the targets move further away, in which case an adaptation of the task assignment is required, as discussed in the sequel.

D. Overall Framework

As summarized in Alg. 3 and illustrated in Fig. 2, the proposed framework is distributed with the following steps.

1) *Online Execution:* Starting from $t = 0$ and under Assump. 2, each target $m \in \mathcal{M}$ and hub $\nu \in \hat{\mathcal{V}}$ is assigned to the closest available agent. Alg. 2 is then applied until a stable solution is reached or a given planning time expires, yielding the plan τ_n for each agent. The local data stored at agent $n \in \bar{\mathcal{N}}$ at time $t \geq 0$ is given by:

$$\Omega_n(t) \triangleq \left(\tau_n, \{\hat{\mathbf{y}}_m(t), \forall m \in \mathcal{M}_n\}, \{\xi_\nu, \forall \nu \in \hat{\mathcal{V}}_n\} \right), \quad (28)$$

which includes the current plan, target state estimates, and hub timers. Each active agent $n \in \bar{\mathcal{N}}$ formulates and solves the NMPC problem in (3) using its local data and horizon T_c , producing the control sequence $\mathbf{u}_n$ and monitoring action $\mathbf{s}_n$. The agent then executes the next control input, obtains new measurements for the assigned targets and hubs, updates $\{\hat{\mathbf{y}}_m(t)\}$ by (9) and $\{\xi_\nu\}$. The controls and actions are followed

until (27) is satisfied and the NMPC is re-solved. During execution, tasks in τ_n are completed, i.e., the uncertainty of target $m \in \mathbf{m}_n$ stays below $\bar{\gamma}_m$ and hub $\nu \in \mathbf{m}_n$ is monitored for duration d_ν . If local plans become infeasible or better plans are found, the shuffle, give, or swap operations are performed locally as in Alg. 2. This process repeats until termination. To guarantee real-time performance, the two layers run at different frequencies. Distributed task coordination is purely event-triggered and is invoked only when agents satisfy (21), complete a monitoring task (27b)-(27c), or encounter local NMPC infeasibility. The motion-planning layer operates at a fixed sampling period, and the nonlinear trajectory optimization is re-solved only when (27) is violated. By decoupling the discrete monitoring actions and exploiting the trace-polynomial SQP, the NMPC solution time remains within the sampling period.

2) *Dynamic Adaptation*: Due to uncertain target movements and sensing disturbances, failures may occur during both task coordination and motion planning. First, as described in Sec. III-B, the local plan τ_n may remain infeasible even after a shuffle operation, e.g., when two targets are both close to intersections. Then Alg. 2 is triggered to identify give or swap operations with neighboring agents; by the objective in (23), an inactive agent may be recruited to restore feasibility under Assump. 1. Second, the NMPC problem in (3) may become infeasible, i.e., no control inputs can maintain the assigned uncertainty bounds, due to the agent dynamics or unexpected target maneuvers. The affected agent then reduces its reference velocity v_i used in the transition-time estimate of TSP-TW, which inflates the required duration and renders the current local schedule infeasible, i.e., $F_{\tau_n} \geq 1$. Since Alg. 2 optimizes $C(\hat{\tau})$ with feasibility as the highest priority, this causes a new coordination round until global feasibility $F_{\hat{\tau}} = 0$ is restored, with a recruited agent if needed.

Theorem 4. *Under Assumptions 1-2 and Theorem 3, the receding-horizon solution of Problems 1-3 provides a distributed solution to the original problem in (8).*

Proof. Prob. 1 provides intention-aware predictions for each target, making the uncertainty and hub constraints in (8) evaluable over finite horizons. Based on these predictions, Prob. 2 computes a distributed task allocation; by Assump. 2 and Theorem 3, it converges in finite steps to a Nash-stable feasible plan with $F_{\hat{\tau}} = 0$, while reducing the active fleet size through the cost $C(\hat{\tau})$. The resulting schedules are executed by Prob. 3, whose event-triggered NMPC explicitly enforces $\det(\tilde{\Sigma}_m(t)) < \gamma_m$ and $\xi_\nu(t) \leq \hat{T}_\nu$ over the prediction horizon. If execution becomes locally infeasible because of disturbances or intention shifts, the event-triggered logic re-invokes Prob. 2; Assump. 1 and Assump. 2 then ensure that a feasible global schedule is recovered. Therefore, the closed-loop interaction of intention inference, distributed coordination, and ENMPC yields a distributed solution to (8). $\square$

3) *Complexity*: The overall framework combines intention inference, task coordination, and trajectory generation. For intention-aware prediction, translating an LTL formula into an NBA has worst-case complexity $\mathcal{O}(2^{2^{|\varphi|}})$, while the as-

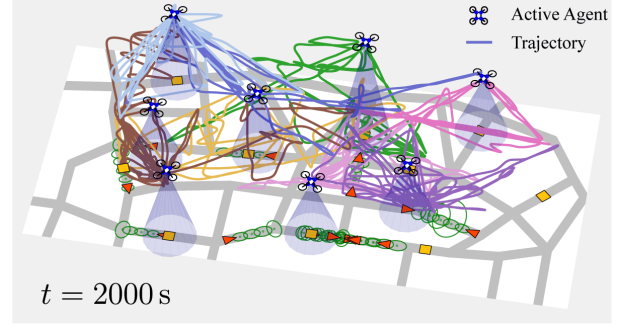


Fig. 10. Complete trajectories of all active agents at $t = 2000$ s.

sociated PBA over the road network has $\mathcal{O}(|\mathcal{V}||Q_j|)$ states. Since this construction is offline, the online cost is dominated by shortest-path queries on the fixed PBA graph, requiring $\mathcal{O}(|\mathcal{V}||Q_j|\log(|\mathcal{V}||Q_j|))$ per update. Prob. 2 remains NP-hard [41], but Alg. 2 restricts give and swap operations to local neighborhoods, so each agent solves only a local TSP-TW instance over $L_n \ll M$ tasks, yielding worst-case cost $\mathcal{O}(L_n^2 2^{L_n})$ per local solve and finitely many communication rounds by Theorem 3. Finally, after fixing the discrete monitoring action, Prob. 3 is solved by SQP, whose dominant factorization cost per iteration is bounded by $\mathcal{O}(T_c d^3)$, with an additional linear overhead in T_c and C_n for evaluating the EKF recursion and determinant constraints.

IV. NUMERICAL EXPERIMENTS

Extensive numerical studies are presented in this section. The proposed method is implemented in Python3 and tested on a laptop with an Intel Core i7-1280P CPU. Solvers from OR-Tools [34] and CasADi [42] are adopted. Setup details and simulation videos can be found in the supplementary files.

A. Simulation Setup

As shown in Fig. 1 and 10, the road network spans $38\text{ m} \times 17\text{ m}$, with 15 dynamic targets, 10 static hubs, and 25 agents. The dynamic targets follow double-integrator dynamics with zero-mean Gaussian process noise [6], [7]. Each agent mimics fixed-wing dynamics at varying heights, with a standard range-bearing sensing model [6], [7]. Key parameters include: the uncertainty bound for dynamic targets $\gamma_m = 10^3$, unknown speed below $v = 0.4\text{ m/s}$, monitoring duration for static hubs $d_\nu = 2\text{ s}$, and the monitoring interval $T_\nu = 100\text{ s}$ in (8). The NMPC planning horizon is $T_c = 20$, with a sampling period $\Delta t = 0.5\text{ s}$, and the triggering condition for NMPC is $\delta_w = 0.1$ in (27a). Further details, including full model definitions, parameter values, Jacobians, and the noise-distance functions, are provided in the supplementary material.

Additionally, denote the indices of dynamic targets by target_{0-14} and static hubs by target_{15-24} . The static hubs are classified to 7 types by type_{0-6} . A total of 6 temporal tasks for the dynamic targets are considered, each of which is expressed as LTL formulas below:

$$\begin{aligned} \varphi_1 &= \square((\text{target}_{16} \Rightarrow \diamond \text{target}_{17}) \wedge (\text{target}_{17} \Rightarrow \diamond \text{target}_{16})), \\ \varphi_2 &= \square(\text{type}_0 \Rightarrow (\neg \text{type}_2 \cup \text{type}_1)), \\ \varphi_3 &= (\square \diamond \text{target}_{19}) \wedge (\square \diamond \text{target}_{20}) \wedge (\square \diamond \text{target}_{21}), \end{aligned}$$

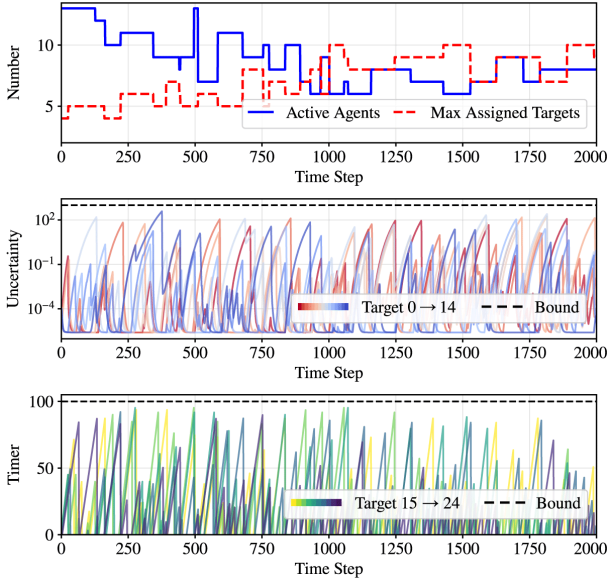


Fig. 11. **Top**: Evolution over time of the average number of active agents (in blue) and the maximum number of targets assigned to each agent (in red); **Middle**: Evolution over time of the uncertainties associated with dynamic targets; **Bottom**: Evolution over time of the timers for static hubs.

$$\begin{aligned}\varphi_4 &= (\Box \Diamond \text{type}_3) \wedge (\Box \Diamond \text{type}_4) \wedge (\Box \Diamond \text{type}_5), \\ \varphi_5 &= \Box((\text{target}_{15} \Rightarrow \Diamond \text{target}_{17}) \wedge (\text{target}_{17} \Rightarrow \Diamond \text{target}_{19}) \\ &\quad \wedge (\text{target}_{19} \Rightarrow \Diamond \text{target}_{15})), \\ \varphi_6 &= \Box \Diamond \text{type}_2;\end{aligned}$$

where each dynamic target is assigned one task and acts according to the optimal plan. Initially, the predicted intention for each target is uniform, and the weights at intersections with K roads are set uniformly to $1/K$.

B. Simulation Results

1) *Overall Performance*: Fig. 1 and 10 show two snapshots of the simulation result at $t = 1730$ s and $t = 2000$ s, respectively. Agents switch between *active* and *inactive* to monitor multiple targets using on-board cameras with limited sensing range. The global statistics are summarized in Fig. 11. At the beginning of the simulation, 13 agents are required to maintain the monitoring-quality constraints for 25 targets due to insufficient intention estimates. As observations accumulate and the Bayesian intention posteriors are updated, the number of active agents gradually decreases and stabilizes at 8 by $t = 2000$ s. Consistently, the frequency of task replanning diminishes over time: within the first 1000 s, re-planning is triggered approximately every 55 steps, whereas in the subsequent 1000 s it occurs every 111 steps. The intention-inference results are reported in Fig. 5. Starting from a uniform prior, the probability of the inferred task intention for each target progressively concentrates and converges to the ground truth. In parallel, the intersection weights along the road-network, which are used for intention-aware trajectory prediction, evolve from a uniform initialization to directional preferences consistent with the assigned task as validated in Fig. 12. For dynamic targets, the evolution of $\det(\Sigma_m)$ remains below the prescribed upper bound at all times; for static hubs, the timer variable $\xi_{\nu,2}$ stays within its admissible

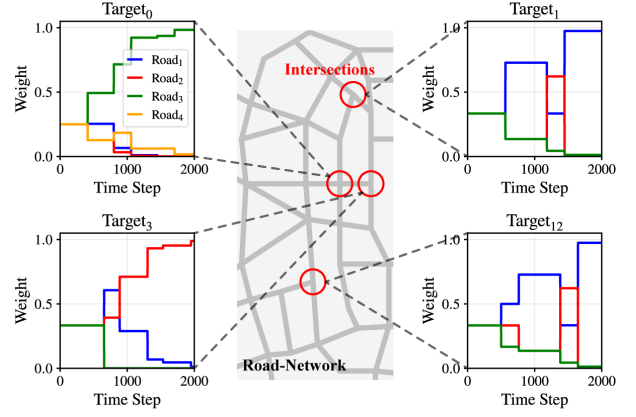


Fig. 12. The evolution of weights at intersections for four dynamic targets over time, which converges along with the intention predictions.

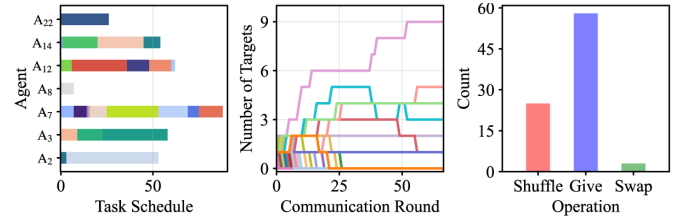


Fig. 13. Results of the distributed task assignment process and its results. **Left**: task schedule for each active agent; **Middle**: number of assigned targets for each agent over the local communication round; **Right**: count of the local operation for the distributed task assignment.

range, as shown in Fig. 11. These results confirm that the intention-aware coordination and the event-triggered controller jointly enforce the uncertainty constraints at all time.

2) *Efficiency Analyses*: The distributed task assignment routine takes on average 0.02 s per agent and yields an average of 3.1 targets per active agent; the fleet has 8.6 active agents on average over the entire run. A representative coordination episode at $t = 1730$ s is depicted in Fig. 13: the local operation process converges to a stable assignment in 70 communication rounds, during which 58 give operations, 3 swap operations, and 25 shuffle operations are executed. The resulting schedule deploys 7 agents for the then-active target set. The average computation time per call of TSP-TW and TSP-TW-MAX is 0.08 ms. On the motion-control layer, the event-triggered NMPC is activated on average 342 times per agent over the entire simulation; each invocation takes approximately 0.06 s. Fig. 14 reports a representative SQP solve at $t = 778$ s. The left panel shows the trajectory update from the initial iterate to the final local trajectory, while the right panel shows the stage-wise defect norms associated with the linearized dynamics and EKF evolution constraints. The defect norms decrease monotonically over the SQP iterations and fall below the prescribed tolerance 10^{-3} within 10 iterations.

The scalability of the intention-inference module in Alg. 1 is further evaluated and shown in Fig. 15. The left panel reports the mean time horizon required for posterior convergence under different product automaton sizes and different numbers of candidate tasks. The convergence time increases with the size of product automaton $\{\mathcal{P}_j\}$ as larger road networks induce more complex reachability structures. The right panel reports the online computation time for one posterior update of

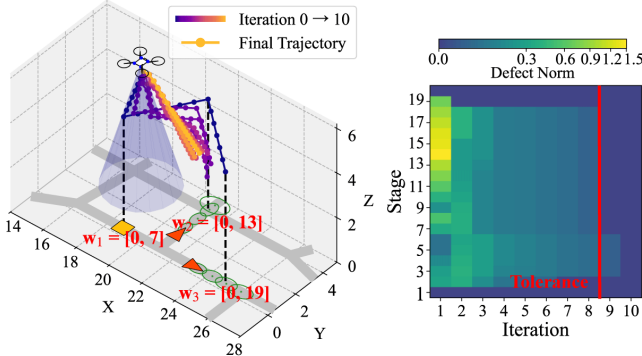


Fig. 14. **Left:** SQP trajectory iterations for agent 19 at $t = 778$ s. **Right:** Stage-wise defect norms of the linearized dynamics and EKF uncertainty constraints over SQP iterations.

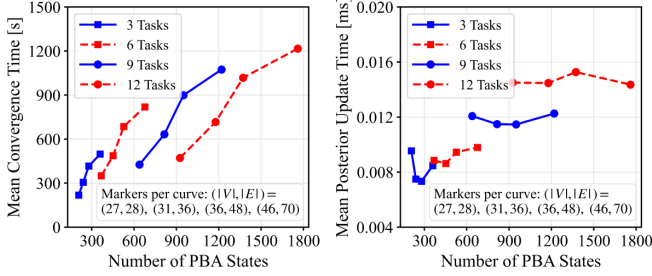


Fig. 15. Mean convergence time (**Left**) and online posterior update time (**Right**) of the intention-inference module under different PBA state counts and numbers of candidate tasks. The marked curves correspond to 3, 6, 9, and 12 candidate tasks, while the road-network sizes are $(|\mathcal{V}|, |\mathcal{E}|) \in \{(27, 28), (31, 36), (36, 48), (46, 70)\}$ from left to right.

intention $\{\chi_m\}$. Since the PBAs are constructed offline, online inference only requires lightweight reachability queries over the precomputed automata. Consequently, the mean posterior update time remains below 0.016 ms even for the largest tested case, which contains 12 candidate tasks and a road network with 46 nodes and 70 edges. This runtime is negligible compared with the average ENMPC solve time, indicating that intention inference does not dominate the online computation.

C. Comparison with Baselines

To further validate the proposed framework, denoted by **Ours**, a quantitative comparison is conducted against six baselines: (I) **w/o Intention**, where the intention-prediction layer for dynamic targets is skipped and the weights at intersections are frozen; (II) **w/o Bound**, where the bound constraints in (24) are removed and the objective is modified to minimize the summed uncertainties of all assigned targets; (III) **NMPC**, which solves a nonlinear MPC formulation [37] using CasADi [42] instead of the SQP-based ENMPC; (IV) **MVRP-TW**, a centralized, time-triggered multi-vehicle routing problem with time-window constraints [21], solved using OR-Tools [34]; (V) **RVI-DS**, a distributed forward reduced value iteration method that couples target prediction and task coordination with motion primitives $v \in \{0, 0.1, 0.25\}$ m/s and $\omega \in \{0, \pm\pi/2, \pm\pi\}$ rad/s, and performs online motion planning via NMPC [4], [5], [7]; and (VI) **Greedy-DS**, a distributed myopic search method that optimizes only the next step using the same motion primitives [4], [14]. The compared metrics are the success rate SR , defined as the fraction of the 2000 time steps in which all target uncertainties remain within

TABLE I
COMPARISONS WITH BASELINE METHODS (OVER 20 RUNS)

Method	SR [%]	t_{sk} [s]	t_{mt} [s]	N_{ag}	N_{ed}
w/o Intention	100.0	0.04	0.08	13.3	12.0
w/o Bound	82.4	0.03	0.02	10.9	11.0
NMPC	98.1	0.02	0.49	9.1	9.0
MVRP-TW	96.8	0.31	0.07	13.5	14.0
RVI-DS	67.3	6.27	0.12	18.9	19.0
Greedy-DS	13.9	0.14	0.13	18.0	20.0
Ours	100.0	0.02	0.06	8.6	8.0

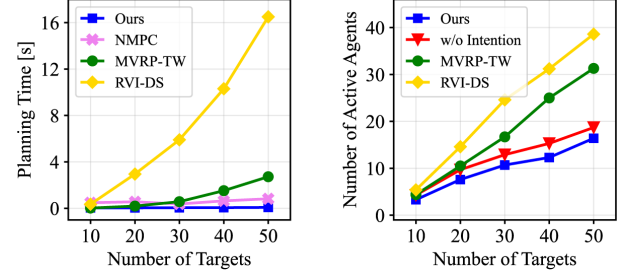


Fig. 16. Comparison of baselines over the planning time (**Left**) and number of active agents (**Right**) under increasing numbers of targets (10 to 50).

bounds, the average task coordination time t_{sk} , the average motion planning time t_{mt} , the average number of active agents N_{ag} , and the final number of active agents N_{ed} .

The results are summarized in Table I. The proposed method attains $SR = 100\%$ with the smallest average and final fleet sizes, namely $N_{ag} = 8.6$ and $N_{ed} = 8.0$, while keeping $t_{sk} = 0.02$ s and $t_{mt} = 0.06$ s. The comparison with **w/o Intention** shows the benefit of intention-aware prediction: both methods achieve $SR = 100\%$, but removing intention prediction increases N_{ag} from 8.6 to 13.3. The comparison with **w/o Bound** shows the importance of enforcing uncertainty bounds. Although this variant has the lowest motion-planning time of 0.02 s, its success rate drops to 82.4%. The remaining baselines highlight the computational advantages of the proposed coordination and control architecture. Compared with **NMPC**, the SQP-based ENMPC reduces t_{mt} from 0.49 s to 0.06 s and improves SR from 98.1% to 100%. Compared with the centralized **MVRP-TW** baseline, the proposed distributed task coordination reduces t_{sk} from 0.31 s to 0.02 s and reduces N_{ag} from 13.5 to 8.6. The primitive-based baselines are less effective in this setting: **RVI-DS** requires 6.27 s for task coordination and 18.9 active agents on average, while **Greedy-DS** attains only $SR = 13.9\%$ and activates all 20 agents at the final time step. These results indicate that intention-aware prediction, distributed task coordination, and ENMPC jointly improve monitoring reliability and efficiency.

Further comparisons with selected baselines as the number of targets increases from 10 to 50 are shown in Fig. 16. The proposed method keeps the average planning time well below 0.2 s across all tested target counts. In contrast, **NMPC** remains below 1.0 s but is consistently slower, **MVRP-TW** increases to about 2.7 s at 50 targets, and **RVI-DS** exceeds 10 s at larger scales because of its large discrete search space. The active-agent curves show a similar trend. Among the plotted methods, the proposed method uses the fewest active agents

TABLE II
SENSITIVITY ANALYSIS OVER KEY PARAMETERS

Parameters	SR [%]	t_{p1} [s]	N_{ag}	N_{ed}	N_{sk}
Nominal*	100.0	0.08	8.6	8.0	37.0
M	10	100.0	0.05	3.3	2.0
	50	100.0	0.12	12.4	13.0
γ_m	1×10^2	95.6	0.24	13.0	12.0
	5×10^2	96.8	0.17	10.5	11.0
v_n	0.2	100.0	0.06	9.7	9.0
	0.3	100.0	0.09	7.8	6.0

* Nominal setting: $M = 25$, $\gamma_m = 1 \times 10^3$, and $v_n = 0.25$ m/s.

TABLE III
ROBUSTNESS UNDER NON-IDEAL CONDITIONS

Metric	η_{ir}		η_{dc}	
	33.3%	66.7%	5.0%	10.0%
SR [%]	100.0	100.0	100.0	100.0
N_{ag}	9.3	10.7	8.8	9.1
N_{ed}	10	12	9	10

and grows slowly with the target count, whereas **w/o Intention** uses about 2–3 more agents, **MVRP-TW** requires substantially larger fleets, and **RVI-DS** approaches 40 active agents at 50 targets. Overall, the scaling results show that the proposed framework maintains fast online planning and moderate fleet size as the monitoring workload increases.

D. Sensitivity and Robustness Analysis

Sensitivity with respect to the number of dynamic targets, the uncertainty bound, and the agent speed are evaluated and summarized in Table II. Note that t_{p1} denotes the average total planning time, including task coordination and motion planning; N_{ag} , N_{ed} denote the average and final number of active agents; and N_{sk} denotes the number of task coordinations. Under the nominal setting, the proposed framework attains $SR = 100\%$ with $t_{p1} = 0.08$ s, $N_{ag} = 8.6$, and $N_{ed} = 8.0$. Increasing M from 10 to 50 preserves 100% success rate and keeps the planning time less than 0.12 s, while N_{ag} increases from 3.3 to 12.4 as the monitoring tasks grow. The uncertainty bound has a stronger effect on both planning and efficiency. When γ_m decreases from the nominal value to 500 and 100, the success rate remains above 95%, but t_{p1} increases to 0.17 s and 0.24 s. The corresponding increases in N_{ag} and N_{sk} indicate that tighter uncertainty bounds require more frequent coordination and larger active fleets. The agent-speed cases show a complementary trade-off. Slower agents require more active agents, whereas faster agents reduce N_{ag} to 7.8 and N_{ed} to 6.0. These results show that the framework maintains high success rates and sub-second planning time across parameter variations.

To further evaluate robustness under non-ideal conditions, the parameters η_{ir} and η_{dc} are introduced to represent: the proportion of irrational dynamic targets, and the detection failure rate for static hubs. Table III reports the corresponding performance under target-behavior mismatch and detection failures. The proposed method maintains $SR = 100\%$ in

all tested cases. As η_{ir} increases to 33.3% and 66.7%, N_{ag} increases to 9.3 and 10.7. Similarly, when η_{dc} increases to 5.0% and 10.0%, N_{ag} to 8.8 and 9.1. These results suggest that the proposed framework can tolerate target-behavior mismatch and detection failures by activating additional agents.

V. CONCLUSION

This paper presented a distributed online framework for joint task allocation and motion planning in large-scale road-network monitoring. The method combines intention-aware prediction, event-triggered ENMPC, and local coordination to enforce explicit uncertainty bounds while reducing the number of active agents. Future work will address several limitations of the current formulation. First, the target behavior is modeled through rational task execution, where a natural extension is to more general game-theoretic settings with strategic or adversarial targets. Second, the current design assumes that each target can be monitored by a single agent, whereas some targets may require collaborative sensing via different sensors. Finally, the intention module is purely model-based, which motivates future integration of learning-based prediction with formal safety guarantees.

REFERENCES

- [1] J. Fu and U. Topcu, “Synthesis of joint control and active sensing strategies under temporal logic constraints,” *IEEE Transactions on Automatic Control*, vol. 61, no. 11, pp. 3464–3476, 2016.
- [2] M. Tzes, N. Bousias, E. Chatzipantazis, and G. J. Pappas, “Graph neural networks for multi-robot active information acquisition,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 3497–3503.
- [3] J. Fu, Z. Wang, and J. Chen, “Coordinated multi-agent patrolling with state-dependent cost rates: Asymptotically optimal policies for large-scale systems,” *IEEE Transactions on Automatic Control*, 2024.
- [4] N. Atanasov, J. Le Ny, K. Daniilidis, and G. J. Pappas, “Information acquisition with sensing robots: Algorithms and error bounds,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 6447–6454.
- [5] P. Dames, M. Schwager, V. Kumar, and D. Rus, “A decentralized control policy for adaptive information gathering in hazardous environments,” in *IEEE Conference on Decision and Control (CDC)*, 2012, pp. 2807–2813.
- [6] Y. Kantaros, B. Schlotfeldt, N. Atanasov, and G. J. Pappas, “Asymptotically optimal planning for non-myopic multi-robot information gathering,” in *Robotics: Science and Systems*, 2019, pp. 22–26.
- [7] B. Schlotfeldt, D. Thakur, N. Atanasov, V. Kumar, and G. J. Pappas, “Anytime planning for decentralized multirobot active information gathering,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 1025–1032, 2018.
- [8] T. H. Chung, J. W. Burdick, and R. M. Murray, “A decentralized motion coordination strategy for dynamic target tracking,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2006, pp. 2416–2422.
- [9] J. Chen, M. Abugurain, P. Dames, and S. Park, “Distributed multi-robot multi-target tracking using heterogeneous limited-range sensors,” *IEEE Transactions on Robotics*, pp. 1–18, 2025.
- [10] J. Wakulicz, K. M. B. Lee, T. Vidal-Calleja, and R. Fitch, “Homotopic information gain for sparse active target tracking,” *IEEE Transactions on Robotics*, vol. 42, pp. 1577–1590, 2026.
- [11] A. A. Tziola and S. G. Loizou, “A formal framework for multi-agent task planning,” *IEEE Transactions on Automatic Control*, vol. 71, no. 1, pp. 630–637, 2026.
- [12] P. Tokekar, V. Isler, and A. Franchi, “Multi-target visual tracking with aerial robots,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2014, pp. 3067–3072.
- [13] L. Zhou and P. Tokekar, “Sensor assignment algorithms to improve observability while tracking targets,” *IEEE Transactions on Robotics*, vol. 35, no. 5, pp. 1206–1219, 2019.

[14] L. Zhou and V. Kumar, "Robust multi-robot active target tracking against sensing and communication attacks," *IEEE Transactions on Robotics*, vol. 39, no. 3, pp. 1768–1780, 2023.

[15] M. Lu, S. Wang, and M. Guo, "LOMORO: Long-term monitoring of dynamic targets with minimum robotic fleet under resource constraints," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 8368–8375.

[16] H. Girase, H. Gang, S. Malla, J. Li, A. Kanehara, K. Mangalam, and C. Choi, "Loki: Long term and key intentions for trajectory prediction," in *IEEE/CVF International Conference on Computer Vision*, 2021, pp. 9803–9812.

[17] L. Xin, P. Wang, C. Y. Chan, J. Chen, S. E. Li, and B. Cheng, "Intention-aware long horizon trajectory prediction of surrounding vehicles using dual lstm networks," in *IEEE International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 1441–1446.

[18] L. Huang, J. Zhen, J. Lygeros, and F. Dörfler, "Robust data-enabled predictive control: Tractable formulations and performance guarantees," *IEEE Transactions on Automatic Control*, vol. 68, no. 5, pp. 3163–3170, 2023.

[19] J. Berberich, J. Köhler, M. A. Müller, and F. Allgöwer, "Data-driven model predictive control with stability and robustness guarantees," *IEEE Transactions on Automatic Control*, vol. 66, no. 4, pp. 1702–1717, 2020.

[20] S. Liu, N. Atanasov, and S. Koga, "MATT-Diff: Multimodal active target tracking by diffusion policy," in *Annual Learning for Dynamics and Control Conference (LADC)*, 2026.

[21] S. Wang, Y. Kantaros, and M. Guo, "Uncertainty-bounded active monitoring of unknown dynamic targets in road-networks with minimum fleet," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 4584–4590.

[22] M. Guo, J. Tumova, and D. V. Dimarogonas, "Communication-free multi-agent control under local tasks and relative-distance constraints," *IEEE Transactions on Automatic Control*, 2016.

[23] M. Kloetzer and C. Belta, "A fully automated framework for control of linear systems from temporal logic specifications," *IEEE Transactions on Automatic Control*, vol. 53, no. 1, pp. 287–297, 2008.

[24] Y. Sung, A. K. Budhiraja, R. K. Williams, and P. Tokekar, "Distributed assignment with limited communication for multi-robot multi-target tracking," *Autonomous Robots*, vol. 44, no. 1, pp. 57–73, 2020.

[25] M. Athans, "On the determination of optimal costly measurement strategies for linear stochastic systems," *Automatica*, vol. 8, no. 4, pp. 397–412, 1972.

[26] M. L. Overton, "On minimizing the maximum eigenvalue of a symmetric matrix," *SIAM Journal on Matrix Analysis and Applications*, vol. 9, no. 2, pp. 256–268, 1988.

[27] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.

[28] C. Baier and J.-P. Katoen, *Principles of Model Checking*. MIT press Cambridge, 2008.

[29] B. Yordanov, J. Tumova, I. Cerna, J. Barnat, and C. Belta, "Temporal logic control of discrete-time piecewise affine systems," *IEEE Transactions on Automatic Control*, vol. 57, no. 6, pp. 1491–1504, 2012.

[30] H. Yoon and S. Sankaranarayanan, "Predictive runtime monitoring for mobile robots using logic-based bayesian intent inference," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 8565–8571.

[31] S. Kalluraya, G. J. Pappas, and Y. Kantaros, "Multi-robot mission planning in dynamic semantic environments," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 1630–1637.

[32] J. Le Ny and G. J. Pappas, "On trajectory optimization for active sensing in gaussian process models," in *IEEE Conference on Decision and Control (CDC)*, 2009, pp. 6286–6292.

[33] S. P. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge university press, 2004.

[34] L. Perron and V. Furnon, "OR-Tools," <https://developers.google.com/optimization/>, Google, 2025, software library.

[35] R. Burkard, M. Dell'Amico, and S. Martello, *Assignment problems: revised reprint*. SIAM, 2012.

[36] L. A. Wolsey and G. L. Nemhauser, *Integer and Combinatorial Optimization*. John Wiley & Sons, 1999, vol. 55.

[37] F. Allgöwer and A. Zheng, *Nonlinear Model Predictive Control*. Birkhäuser, 2012, vol. 26.

[38] Y. Kang and J. K. Hedrick, "Linear tracking for a fixed-wing uav using nonlinear model predictive control," *IEEE Transactions on Control Systems Technology*, vol. 17, no. 5, pp. 1202–1210, 2009.

[39] R. A. Horn and C. R. Johnson, *Matrix Analysis*. Cambridge university press, 2012.

[40] A. Hakobyan and I. Yang, "Wasserstein distributionally robust control of partially observable linear stochastic systems," *IEEE Transactions on Automatic Control*, vol. 69, no. 9, pp. 6121–6136, 2024.

[41] M. Polacek, R. F. Hartl, K. Doerner, and M. Reimann, "A variable neighborhood search for the multi depot vehicle routing problem with time windows," *Journal of Heuristics*, vol. 10, pp. 613–627, 2004.

[42] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.



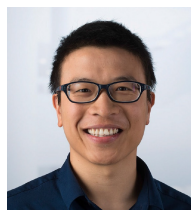
Shuaikang Wang (M'23) received the B.E. degree in Robotics Engineering in 2023 from Peking University, Beijing, China. He is currently a senior graduate student in the School of Advanced Manufacturing and Robotics, Peking University, China. His current research interests include decision-making, learning, and control in robotic systems. He was the finalist for the Best Multi-Robot Systems Paper Award at ICRA 2024.



Jianyang Xue received the B.S. degree in theoretical and applied mechanics from Fudan University, Shanghai, China. He is currently a graduate student in the School of Advanced Manufacturing and Robotics, Peking University, Beijing, China. His research interests include planning and control for multi-robot systems.



Yiannis Kantaros (M'12) received the diploma in electrical and computer engineering from the University of Patras, Patras, Greece, in 2012, and the M.Sc. and Ph.D. degrees in mechanical engineering from Duke University, Durham, NC, USA, in 2017 and 2018, respectively. He is an Assistant Professor with the Department of Electrical and Systems Engineering, Washington University in St. Louis (WashU), St. Louis, MO, USA. Prior to joining WashU, he was a Postdoctoral Associate with the Department of Computer and Information Science, University of Pennsylvania, Philadelphia, PA, USA. His current research interests include machine learning, distributed control and optimization, and formal methods with applications in robotics. Dr. Kantaros was the recipient of the 2017–18 Outstanding Dissertation Research Award from the Department of Mechanical Engineering and Materials Science, Duke University, and a 2024 NSF CAREER Award.



Meng Guo (M'11) received the M.Sc. degree (2011) in System, Control, and Robotics and the Ph.D. degree (2016) in Electrical Engineering from KTH Royal Institute of Technology, Sweden. He was a postdoctoral associate with the Department of Mechanical Engineering and Materials Science, Duke University, USA. During 2018–2021, he worked as a senior research scientist on Reinforcement Learning and Planning at the Bosch Center for Artificial Intelligence (BCAI), Germany. Since 2022, he is an assistant professor at the School of Advanced Manufacturing and Robotics, Peking University, China. His main research interests include task and motion planning for robotic systems.